

jwt

Signing and Verifying JSON Web Tokens in Go

Gerasimos (Makis) Maropoulos

Author of the jwt package

Creator of Iris

FIRST EDITION

Contents

Preface

Who this is for

What this book is not

How the code in here was checked

The shape of the book

A note on the word secure

1 What a Token Is

Three segments and two dots

Why base64url and not base64

Taking a token apart in Go

What the signature covers

What verifying proves, and what it does not

The compact form is not encryption

2 Signing and Verifying

The two functions

A complete program

Why the algorithm is a parameter

What compareHeader actually does

What a VerifiedToken gives you

Decode versus Verify

The errors a failed verification returns

A custom header replaces the whole header

What Sign accepts as claims

The size ceiling

3 Claims

The Claims struct, field by field

Custom claims in a struct of your own

Combining custom claims with standard ones

The sign-option shorthands

How Merge splices

The Audience type

The second-chance parser

Required fields

4 Expiry and Time

The three time claims

Why zero means absent

Setting a lifetime with MaxAge

Requiring an expiry

Clock skew and the order of validators

Leeway is stricter than it sounds

Replacing the clock in tests

Reading time off a verified token

Choosing lifetimes

5 Choosing an Algorithm

The six families

Symmetric or asymmetric

Key types and wrong keys

Sizes and cost

Determinism and golden tests

The none algorithm

Writing your own Alg

What to pick

6 Keys and Rotation

The flag day problem

The kid header and the Keys registry

Signing and verifying through the registry

ValidateHeader binds the kid to an algorithm

HMAC in a registry

Enriching a token the registry signed

Keys from a configuration file

Loading keys from PEM

MustLoadHMAC will not tell you the file was missing

The registry is a plain map

A rotation in four steps

7 JWKS

What a key set is

Publishing your own keys

Generating a single JWK

Fetching somebody else's keys

What the fetch refuses to do

KeySet keeps the keys current

AWS Cognito

8 Validators

The interface and its function form

The chaining rule

Expected, field by field

Plain, for a payload that is not JSON

Leeway, Skew, Future and RequireExpiry

Collecting validators with Validators

Writing your own

Classify, and what to tell the client

9 Revoking Tokens

Why a token cannot be taken back

Short lifetimes come first

The blocklist

Choosing the key

Tokens with no expiry

Where it sits in the validator chain

A distributed blocklist

Revoking a refresh token with its access token

10 Tokens over HTTP

Where a token lives in a request

The extractors

What FromHeader gets right

A middleware that compiles

Why the context key is not a string

From error to status code

Access and refresh tokens

The response shape

A refresh endpoint

11 Encrypting the Payload

What signing does not do

GCM and the two functions it returns

Signing and verifying an encrypted token

The order of operations

Additional authenticated data

The nonce and the error you do not get

Encryption in the key registry

This is not JWE

What it costs

When to keep the data on the server instead

12 Sharp Edges

Names that read the wrong way round

Defaults that are quieter than you want

The package-level variables

Keys is a plain map

Merge splices JSON and can emit a duplicate key

The unsafe string conversion

What Enrich used to be

Behaviour that changed in v0.2.0

13 Testing

The suite that never opens a socket

Controlling time

Keys for tests

Faking the network

A fake that rotates

Golden tests

Table-driven tests for the error paths

Testing that something is refused

Replacing Marshal and Unmarshal

The race detector

14 **Performance**

The numbers on one machine

Where the time goes in a verification

Precomputed headers

Bytes to string without a copy

Merge splices raw JSON

Marshal passes bytes through

Base64Decode trims instead of padding

Measuring your own usage

What not to optimise

The comparison benchmarks

Epilogue

The pattern worth taking away

What this package is not going to do for you

Contributing

Thanks

Preface

A JSON Web Token is three pieces of base64url text joined by dots. That is the whole format. Everything difficult about tokens happens around that format: which algorithm you pinned, which key you verified with, whether the claim you read was the one that was signed, and what you do on the day a key has to be replaced while requests keep arriving.

This book is about `github.com/kataras/jwt`, a JWT and JWKS implementation for Go with no dependencies. It covers what the package does, and it spends at least as long on why the API is shaped the way it is. An API you can predict is worth more than one you have to look up, and the only way to make it predictable is to say what it was worried about.

Who this is for

You write Go, and something in your system hands out tokens or accepts them. You do not need to have read RFC 7519. Chapter 1 builds the format from nothing, and every chapter after that assumes only what came before it.

If you already know JWT well, skip to Chapter 5 for the algorithm trade-offs, Chapter 7 for key rotation, or Chapter 12, which is the honest list of this package's sharp edges.

What this book is not

It is not a reference. The reference is `go doc`, and it is generated from the source, which means it cannot drift. This book is the part `go doc` cannot give you: the order to do things in, the reason a default is what it is, and the failure that made somebody choose it.

It is also not a security course. It will tell you that verifying a signature proves who wrote a token and nothing else about whether you should trust its contents, and it will show you the code that gets that wrong. It will not teach you threat modelling.

How the code in here was checked

Every signature quoted in this book was compared against the package source, not remembered. That sounds like a low bar. It is not: the package's own documentation contained a dozen examples that could not compile, including four that showed the wrong number of return values and three that referenced a type which does not exist. Prose is not compiled, so nothing catches this for you.

Where a chapter says a call returns a particular error, that error was produced and read. Where it quotes a benchmark number, that benchmark was run on the machine named beside it.

The shape of the book

Chapters 1 to 4 are the parts every user needs: the format, signing and verifying, claims, and time. Chapters 5 to 7 are about keys, which is where real systems spend their difficulty. Chapters 8 to 11 cover validation, revocation, HTTP, and payload encryption. Chapter 12 is the sharp edges. Chapters 13 and 14 are testing and performance.

Read 1 to 4 in order. After that, take what you need.

A note on the word secure

This book avoids calling anything secure. A token is signed, or it is not. A signature verifies under a key, or it does not. A claim was covered by that signature, or it was not. Those are checkable. “Secure” is a summary of somebody’s judgement about a whole system, and a library cannot make that judgement for you.

Where this book does make a recommendation, it says what the recommendation protects against and what it costs. You can then disagree with it on purpose rather than by accident.

CHAPTER 1

What a Token Is

A JSON Web Token is a string with two dots in it. Before any of this package's API makes sense, it is worth taking one apart by hand, because almost every mistake people make with tokens comes from a wrong mental model of what those three pieces are and which of them the signature covers. This chapter builds the format from nothing, decodes a real token character by character, and then states precisely what verifying one does and does not tell you.

Three segments and two dots

Here is a complete token, produced by this package with the shared key `sercrethatmaycontainch@r$`:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImthdGFyYXMifQ.HxPZbhkeH0DtRAOJH_1F0YnbmxJvNQiaLzFHBQ-QLtc
```

Split it on the dots and you get three parts:

Part	Decoded	What it is
`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9`	`{"alg":"HS256","typ":"JWT"}`	The header. Says how the token was signed.
`eyJ1c2VybmFtZSI6ImthdGFyYXMifQ`	`{"username":"kataras"}`	The payload. Your claims.
`HxPZbhkeH0DtRAOJH_1F0YnbmxJvNQiaLzFHBQ-QLtc`	32 raw bytes	The signature.

The first two are JSON documents that have been base64url encoded. The third is not text at all: it is the raw output of a signing function, encoded the same way so it survives being put in a URL or an HTTP header.

That is the entire compact serialization. There is no length prefix, no version marker and no envelope. A parser splits on `.`, expects exactly three parts, and goes from there.

Why base64url and not base64

Standard base64 uses `+` and `/` in its alphabet and pads with `=`. All three characters mean something in a URL, so a token pasted into a query string or a cookie would need escaping, and

something in the chain would eventually escape it twice.

Base64url swaps `+` for `-` and `/` for `_`, and JWT drops the padding entirely. RFC 7515 requires that: the padding carries no information, since the length of the encoded text already tells a decoder how many bytes came in.

Dropping the padding is why decoding a segment is slightly more than a library call. The standard decoder wants input whose length is a multiple of four. This package handles it by removing padding rather than adding it:

```
func Base64Decode(src []byte) ([]byte, error) {
	src = bytes.TrimRight(src, padStr)

	buf := make([]byte, base64.RawURLEncoding.DecodedLen(len(src)))
	n, err := base64.RawURLEncoding.Decode(buf, src)
	return buf[:n], err
}
```

The obvious alternative, appending `=` characters until the length divides by four, is a trap, and this package fell into it for years. Chapter 12 explains what went wrong: the short version is that the functions splitting a token into its three parts (`bytes.Split` for a long time, `bytes.CutLast` / `bytes.Cut` since Go 1.27) hand back a final segment that still owns the whole token's spare capacity, so appending to it writes past the end of the caller's buffer. Trimming has no such problem, needs no allocation, and turned out to be faster.

Taking a token apart in Go

You do not have to do any of that by hand. `Decode` parses a token without checking anything:

```
unverified, err := jwt.Decode(token)
if err != nil {
	return err
}

fmt.Printf("header:  %s\n", unverified.Header)
fmt.Printf("payload: %s\n", unverified.Payload)
```

```
header:  {"alg":"HS256","typ":"JWT"}
payload: {"username":"kataras"}
```

The type it returns is called `UnverifiedToken`, and the name is doing real work. Nothing in that struct has been checked. The signature was decoded but not tested. The header says `HS256` because the sender wrote `HS256`, not because anybody agreed.

Use `Decode` when you need to look at a token before you can verify it: reading the `kid` to work out which key to fetch, or logging which issuer a rejected token claimed. Do not use it to read claims you are going to act on.

What the signature covers

This is the part worth being precise about. The signature is computed over the header and the payload **in their encoded form**, joined by a dot:

```
signature = Sign(key, base64url(header) + "." + base64url(payload))
```

Three consequences follow, and each one is a bug somebody has shipped.

The signature covers the header. An attacker cannot change `alg` without invalidating it. That is what makes the header trustworthy after verification, and it is why this package will carry a verified header forward into an enriched token but never an unverified one.

The signature covers the encoded bytes, not the decoded JSON. Two payloads that parse to the same object can have different signatures, and one signature is valid for exactly one byte sequence. You cannot re-serialize a payload and keep the signature. That is why adding a claim to an existing token means signing a new token, which is what `Enrich` does.

The signature covers nothing else. Not the transport, not the time you received the token, not who sent it to you. A signed token that is valid for somebody else is still a valid token when it arrives from an attacker who stole it.

What verifying proves, and what it does not

When `Verify` returns without an error, you have learned exactly one thing:

> Somebody holding the key you supplied produced these exact header and payload bytes.

That is worth a lot, and it is less than people assume. Here is the same statement turned into the questions it does and does not answer.

Question	Does verification answer it?
Were these bytes produced by a holder of the key?	Yes.
Have the claims been modified since?	No, they have not. Verification proves that.
Is the token within its validity window?	Only because this package also checks `exp` and `nbf`. That is claim validation, not signature verification.
Was it issued for me?	No. Check `aud` yourself, or with `Expected`.
Was it issued by who I think?	No. Check `iss`.
Is the person presenting it the person it was issued to?	No. Nothing in a bearer token can tell you that.
Has it been revoked?	No. See Chapter 9.

The last two are where real incidents come from. A JWT is a bearer credential: possession is the whole authorization. If it leaks, whoever holds it is you, until it expires.

The compact form is not encryption

Base64 is an encoding, not a cipher. Anyone holding a token can read its payload with a one-line command:

```
echo 'eyJ1c2VybmFtZSI6ImthdGFyYXMifQ' | base64 -d
```

```
{"username":"kataras"}
```

People put things in payloads that they would not put in a URL, on the reasoning that the token looks opaque. It is not opaque. Treat the payload as public. Put an identifier in it and look the rest up on your side.

If you genuinely need the payload hidden from the person carrying it, Chapter 11 covers the package's AES-GCM support, along with the cases where it does not help.

Summary

A JWT is three base64url segments separated by dots: a header saying how it was signed, a payload of claims, and a signature over the first two in their encoded form. Base64url exists so the result survives a URL, and JWT drops the padding, which is why decoding trims rather than pads.

Verification proves that a holder of your key produced those exact bytes. It does not prove the token is for you, is from who you expect, is still wanted, or is being presented by its owner. Those are separate checks, and the rest of this book is largely about making them.

The payload is readable by anyone holding the token. Encoding is not encryption.

Further Reading

- RFC 7519: JSON Web Token: the format itself, including the registered claim names.
- RFC 7515: JSON Web Signature: the compact serialization and exactly what the signing input is. Section 5.1 is the one to read.
- RFC 4648 section 5: base64url, and why the padding is optional.
- jwt.io: paste a token in and see it split. Useful for a token you did not produce, and a good demonstration that the payload is public.

CHAPTER 2

Signing and Verifying

Two functions carry almost all of the traffic in this package: `Sign` produces a token, and `Verify` turns one back into claims or refuses to. Everything else is either a variation on those two or a way of choosing the key they use. This chapter covers both in full, and then spends most of its length on one design decision that follows from Chapter 1: the algorithm is something you pass in, and it is never read out of the token you are checking.

The two functions

Here are both signatures, copied from the source rather than from memory:

```
func Sign(alg Alg, key PrivateKey, claims any, opts ...SignOption) ([]byte, error)

func Verify(alg Alg, key PublicKey, token []byte, validators ...TokenValidator)
(*VerifiedToken, error)
```

The first two parameters have the same shape in both, in the same order, because signing and verifying are the same operation seen from two sides. Both return byte slices rather than strings, since a token is going into an HTTP header or a cookie and `Sign` joins three byte slices to build it anyway.

`PrivateKey` and `PublicKey` are declared as `= any`. They are aliases, not distinct types, so the compiler will not stop you handing an `*rsa.PrivateKey` to `Verify`. The price is a runtime failure instead of a build failure: a mismatched key comes back as `ErrInvalidKey`, and passing `"secret"` where `[]byte("secret")` was needed produces `expected a string: jwt: invalid key`. What you get is one function covering HMAC secrets, RSA and ECDSA key pairs and Ed25519 keys with no generics and no per-algorithm entry points. Chapter 5 goes through which key type each algorithm wants.

A complete program

This compiles and runs against the package as it stands. Nothing has been elided.

```
package main

import (
        "fmt"
```

```
"time""github.com/kataras/jwt"
)

type userClaims struct {
	Username string`json:"username"`
	Role     string`json:"role"`
}

funcmain() {
	sharedKey := []byte("sercrethatmaycontainch@r$")

	token, err := jwt.Sign(jwt.HS256, sharedKey,
		userClaims{Username: "kataras", Role: "admin"},
		jwt.MaxAge(15*time.Minute))
	if err != nil {
		log.Fatal(err)
	}

	verified, err := jwt.Verify(jwt.HS256, sharedKey, token)
	if err != nil {
		log.Fatal(err)
	}

	var claims userClaims
	if err := verified.Claims(&claims); err != nil {
		log.Fatal(err)
	}

	fmt.Printf("token: %s\n", token)
	fmt.Printf("user:  %s (%s)\n", claims.Username, claims.Role)
	fmt.Printf("left:  %s\n", verified.StandardClaims.Timeleft())
}
```

```
token:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImthdGFyYXMiLCJyb2xlIjoiYWRtaW4iL
CJpYXQiOjE3ODY4NzQ4NjAsImV4cCI6MTc4Njg3NTc2MH0.6I8OqoqQPJffSoY3NphOfS5ntrYcoLACpYHe58x
UWPk
user:  kataras (admin)
left:  15m0s
```

Three things there are worth naming. `jwt.HS256` appears twice, once on each side, and both are written out by you: that is the subject of the next section. `jwt.MaxAge(15 * time.Minute)` is a `SignOption`, and it sets `exp` and `iat` from the current time, which is why the payload holds two claims the struct never declared (Chapter 3 covers sign options, Chapter 4 covers time). And `verified.Claims(&claims)` unmarshals the payload a second time: `Verify` already parsed it once into `StandardClaims`, but it has no idea what your struct looks like, so your own claims cost one more `json.Unmarshal`.

Why the algorithm is a parameter

A JWT header names its own algorithm. The obvious implementation reads that name and picks the verifier to match. Do not write that implementation, and this package makes it awkward to write by accident.

Consider a service that verifies with an RSA public key and reads `alg` from the token to decide what to do. An attacker takes a genuine token, replaces the header with `{"alg":"HS256","typ":"JWT"}`, and computes an HMAC-SHA256 signature over the new header-and-payload using the RSA public key bytes as the shared secret. The public key is public: it is on your JWKS endpoint, or in a certificate, or in a repository.

The service now does exactly what it was told. The header says HS256, so it runs HMAC-SHA256, and the key it has to hand is the RSA public key. The MAC matches, because the attacker computed it with the same bytes. One asymmetric key pair has become a shared secret everybody already has, and the attacker can mint any claims they like.

That is the algorithm-confusion attack. It has been in the JWT security literature since 2015 and keeps reappearing, because the vulnerable design reads as the reasonable one: the token says how it was signed, so believe it.

This package will not let you believe it. `Verify` takes `alg` as its first argument, and the token's header never chooses the verifier. Pass `jwt.RS256` and an RSA public key, hand the forged token to it, and the result is `ErrTokenAlg`, returned before any signature is computed. The forged signature is never checked against anything, because the header did not survive the earlier test.

The trade-off is that you have to know your own algorithm at the call site. For a service issuing its own tokens that is free. For one consuming tokens from an identity provider that rotates algorithms it is not, and that is what `VerifyWithHeaderValidator` and the `Keys` registry exist for (Chapters 6 and 7). Both still keep the choice explicit: a `HeaderValidator` returns the `Alg` to use, so the selection is code you can read rather than a lookup keyed on attacker-controlled text.

What compareHeader actually does

The check that rejects the forged header is `compareHeader` in `token.go`, and it does not parse JSON at all.

At package initialisation, `token.go` fills a map called `fixedHeaders`, one `fixedHeader` entry per built-in algorithm. Each entry holds three byte slices: `raw`, the header JSON, `encoded`, its base64url form, and `reversed`, the same JSON with the two fields in the opposite order. For HS256, `raw` and `reversed` are `{"alg":"HS256","typ":"JWT"}` and `{"typ":"JWT","alg":"HS256"}`, both 27 bytes. Verification base64-decodes the incoming header and runs `bytes.Equal` against one of them. There is no `alg` field to extract, because nothing is ever extracted.

Two cheap tests decide which form to compare against. A decoded header shorter than 25 bytes is accepted only at exactly 15, the length of `{"alg":"HS256"}`, a header with no `typ` at all. Otherwise, if byte index 2 is `t` the header started `{"typ"` and the reversed form is used. Everything else goes against the standard order. Then `bytes.Equal`, and `ErrTokenAlg` if it differs.

This is strict, and it is worth seeing how strict. Each of these was signed with the correct HS256 key and passed to `jwt.Verify(jwt.HS256, key, token)`:

Header	Result
`{"alg":"HS256","typ":"JWT"}`	accepted
`{"typ":"JWT","alg":"HS256"}`	accepted
`{"alg":"HS256"}`	accepted
`{"alg":"HS256","typ":"jwt"}`	`ErrTokenAlg`
`{"alg":"HS256", "typ":"JWT"}`	`ErrTokenAlg`
`{"typ":"JWT","alg":"HS256","kid":"a"}`	`ErrTokenAlg`

One space and a lowercase `jwt` are both fatal, and so is a `kid`, which is a perfectly ordinary header field. That is a real cost: a token from another implementation that formats its header differently is rejected even though its signature is fine. What you get for it is that the parsing step an attacker would need to influence does not exist. There is no field ordering to confuse, no duplicate-key handling to exploit, no Unicode escape to smuggle a second `alg` through. The comparison is against 27 bytes your process chose at startup.

When you need the flexibility back, supply a `HeaderValidator`:

```
type HeaderValidator func(alg string, headerDecoded []byte) (Alg, PublicKey,
InjectFunc, error)
```

The package-level `CompareHeader` variable holds the strict implementation and is used when you pass none. Replacing it is how `kid` support works, and it is why a validator returns a

`PublicKey` too: having read the header, it can also choose the key.

What a VerifiedToken gives you

`Verify` returns `*VerifiedToken`, which is five fields and one method.

```
type VerifiedToken struct {
        Token          []byte
        Header         []byte
        Payload        []byte
        Signature      []byte
        StandardClaims Claims
}

func (t *VerifiedToken) Claims(dest any) error
```

`Token` is the original bytes you passed in, unchanged. Keep it if you will pass the token onward or key a blocklist entry on it. `Signature` is the decoded signature, 32 bytes for HS256, useful mainly for logging.

`Header` and `Payload` are base64-decoded JSON. For the program above they read `{"alg":"HS256","typ":"JWT"}` and `{"username":"kataras","role":"admin","iat":1786874860,"exp":1786875760}`. This header is trustworthy in a way the one from `Decode` is not, because the signature covered it and the signature verified. `StandardClaims` is a `Claims` struct filled from the payload during verification, with its time claims already checked. Chapter 3 walks its fields.

`Claims(dest)` unmarshals `Payload` into anything JSON can fill: a tagged struct, a `jwt.Map`, a type with its own `UnmarshalJSON`. It calls the package-level `Unmarshal` variable, so replacing that replaces the behaviour everywhere, which is how `UnmarshalWithRequired` is switched on in Chapter 3. It re-parses on every call, so if you need three views of one payload, unmarshal once into a struct holding all three sets of fields.

Decode versus Verify

`Decode` splits a token and base64-decodes the pieces. It checks no signature, no expiry and no algorithm. The result is an `UnverifiedToken`, with `Header`, `Payload` and `Signature` fields, its own `Claims(dest any) error`, and two header helpers that cache what they read:

```
func Decode(token []byte) (*UnverifiedToken, error)

func (t *UnverifiedToken) Alg() (Alg, error)
```

```
func (t *UnverifiedToken) Kid() (string, error)
```

There are two honest uses for this.

Deciding which key to fetch. You cannot verify until you have the key, and the `kid` naming that key is inside the token, so reading it first is unavoidable. What makes it safe is that the value is used only as a lookup, and the verification that follows decides whether the token is real. A `kid` naming a key you do not have produces `ErrUnknownKid`, and a `kid` naming the wrong key produces a signature failure.

Reporting on tokens you rejected. When verification fails and you want to log which issuer the token claimed, `Decode` is the only way to see it. Log it as a claim, not a fact.

Everything else is a mistake waiting for its incident. `unverified.Claims(&user)` followed by trusting `user.Role` is the whole vulnerability, in two lines that look like normal code. If your reason for reaching for `Decode` is that the token comes from a trusted internal source, "internal" describes your network and the payload describes what somebody typed.

The errors a failed verification returns

Verification can fail in about a dozen ways, and services that map them to HTTP status codes end up maintaining a list of sentinels that drifts out of date. `Classify` in `classify.go` collapses them into the five values an application actually branches on:

```
func Classify(err error) ErrorKind

switch jwt.Classify(err) {
case jwt.KindNone:
        // carry on
case jwt.KindExpired:
        http.Error(w, "token expired", http.StatusUnauthorized) // the client can
refresh
default:
        http.Error(w, "unauthorized", http.StatusUnauthorized)
}
```

Here is what the common failures produce. Every row was run.

Failure	Error	`Classify`
`Verify(alg, key, nil)`	`ErrMissing`	`KindMalformed`
Not three dot-separated parts	`ErrTokenForm`	`KindMalformed`
Longer than `MaxTokenSize`	`ErrTokenSize`	`KindMalformed`
Header does not match	`ErrTokenAlg`	`KindInvalid`
Payload altered, or the wrong secret	`ErrTokenSignature`	`KindInvalid`
Key of the wrong type	`ErrInvalidKey`	`KindInvalid`
`exp` in the past	`ErrExpired`	`KindExpired`
`nbf` in the future	`ErrNotValidYet`	`KindExpired`
`iat` in the future	`ErrIssuedInTheFuture`	`KindExpired`
`Expected{Issuer: ...}` did not match	`ErrExpected`	`KindRejected`
A required claim was absent	`ErrMissingKey`	`KindRejected`

`Classify` uses `errors.Is`, so a wrapped error still classifies, and anything it does not recognise comes back as `KindInvalid`, the conservative reading of an unrecognised verification failure. Two categories land somewhere you might not expect: a payload that is not JSON, and a base64 error out of a segment that is not base64 at all, both come back `KindMalformed`. Neither is one of this package's sentinel errors: the first is unexported and the second is a `base64.CorruptInputError`, so `Classify` matches them specially.

`KindExpired` is the one distinction worth surfacing to a client of your own, since refreshing is a fix the client can apply. For anybody else, return the same body for every failing kind. Telling an attacker whether a token was expired, forged or revoked is information given away for nothing.

A custom header replaces the whole header

`SignWithHeader` lets you put your own fields in the header:

```
func SignWithHeader(alg Alg, key PrivateKey, claims any, customHeader any, opts
...SignOption) ([]byte, error)
```

The package's own documentation describes those fields as being merged with the standard ones. They are not. Here is the whole implementation from `token.go`:

```
func createCustomHeader(header any) ([]byte, error) {
	b, err := Marshal(header)
	if err != nil {
		return nil, err
	}

	return Base64Encode(b), nil
}
```

Your value is marshalled and encoded. Nothing adds `alg`, nothing adds `typ`, and whatever you pass is the entire header.

```
token, _ := jwt.SignWithHeader(jwt.HS256, sharedKey, jwt.Map{"sub": "1"},
	jwt.Map{"kid": "key-1"})

unverified, _ := jwt.Decode(token)
fmt.Printf("%s\n", unverified.Header) // {"kid":"key-1"}
```

That token is signed correctly, and `jwt.Verify(jwt.HS256, sharedKey, token)` rejects it with `ErrTokenAlg`, because the strict comparison from earlier finds no match. Restating the standard fields does not save you, for a reason worth knowing about:

```
jwt.SignWithHeader(jwt.HS256, sharedKey, claims,
	jwt.Map{"alg": "HS256", "typ": "JWT", "kid": "key-1"})
// header: {"alg":"HS256","kid":"key-1","typ":"JWT"}
```

`encoding/json` sorts map keys, so `kid` lands between `alg` and `typ` and the byte comparison fails. Raw JSON in the exact standard order fares no better: the header is then `{"alg":"HS256","typ":"JWT","kid":"key-1"}`, still not one of the three forms `compareHeader` knows.

The conclusion is short. Signing with a custom header commits you to verifying with a `HeaderValidator`. `VerifyWithHeaderValidator` takes one, and `Keys.ValidateHeader` is a ready-made one that reads `kid` and picks the key. Chapter 7 covers it.

What Sign accepts as claims

The `claims` parameter is `any`, and it goes through the package-level `Marshal` variable:

```
var Marshal = func(v any) ([]byte, error) {
	if b, ok := v.([]byte); ok {
		return b, nil
	}
```

CHAPTER 3

Claims

A payload is a JSON object, and the names in it are claims. Seven of them are registered by RFC 7519 and this package models those in a struct called `Claims`; everything else is yours. The interesting part is not either half on its own but how the two get joined into one object, because the answer explains a performance choice, a duplicate key you may see in your own tokens, and a parser that will accept a numeric `sub` but refuses a structured one.

The Claims struct, field by field

Here is the whole type from `claims.go`, tags included:

```
type Claims struct {
	NotBefore int64    `json:"nbf,omitzero"`
	IssuedAt  int64    `json:"iat,omitzero"`
	Expiry    int64    `json:"exp,omitzero"`
	ID        string   `json:"jti,omitempty"`
	OriginID  string   `json:"origin_jti,omitempty"`
	Issuer    string   `json:"iss,omitempty"`
	Subject   string   `json:"sub,omitempty"`
	Audience  Audience `json:"aud,omitempty"`
}
```

The three timestamp fields are `omitzero`, not `omitempty`. Under the encoding/json this package used through Go 1.26 the two are the same for a plain `int64`: both omit the Go zero value. They stop agreeing once `Marshal` moves to `encoding/json/v2` (below), because v2's `omitempty` only omits a value that already encodes to an empty JSON value, and a numeric `0` is not one of those. `omitzero` was chosen deliberately to keep an unset `exp` absent from the payload rather than emitting `"exp":0`.

The Go names and the JSON names differ on purpose. The three-letter forms are what goes on the wire, because that is what RFC 7519 registered and what every other implementation looks for. The longer forms are what you read in your own code.

Field	Claim	What it holds
`NotBefore`	`nbf`	Seconds since the epoch before which the token is not valid.
`IssuedAt`	`iat`	Seconds since the epoch at which the token was signed.
`Expiry`	`exp`	Seconds since the epoch after which the token is refused.
`ID`	`jti`	A unique identifier for this token.
`OriginID`	`origin_jti`	The `jti` of the token this one came from.
`Issuer`	`iss`	Whoever minted the token.
`Subject`	`sub`	The principal the token is about, usually a user.
`Audience`	`aud`	Who the token is for, as an `Audience`.

Every tag carries `omitempty`, so a zero field is left out of the payload rather than written as `0` or `""`. That is what makes a `Claims` value usable as a partial template, and it is also why validation treats zero as absent. Chapter 4 covers what that means for time.

`origin_jti` is not a registered claim. The other seven are in the IANA JSON Web Token Claims registry. This one is a convention of this package: it links a refresh token to the access token issued alongside it, so revoking one can revoke the other. `SignPair` sets it for you and Chapter 9 covers the revocation side. Send a token carrying `origin_jti` to somebody else's service and expect them to ignore it.

Three methods read the time fields, all covered in Chapter 4:

```
func (c Claims) Age() time.Duration      // Expiry − IssuedAt, the designed lifetime
func (c Claims) ExpiresAt() time.Time    // time.Unix(c.Expiry, 0)
func (c Claims) Timeleft() time.Duration // Expiry − Clock().Unix()
```

Custom claims in a struct of your own

Your own claims are an ordinary Go struct with JSON tags. Nothing in the package needs to know about the type. Pass a value of it to `Sign`, and read it back with `VerifiedToken.Claims`:

```
type sessionClaims struct {
	Username string   `json:"username"`
	Scopes   []string `json:"scopes"`
}

token, err := jwt.Sign(jwt.HS256, sharedKey,
	sessionClaims{Username: "kataras", Scopes: []string{"read", "write"}},
	jwt.MaxAge(15*time.Minute))

verified, err := jwt.Verify(jwt.HS256, sharedKey, token)

var claims sessionClaims
err = verified.Claims(&claims)
```

`Claims` takes a pointer and hands it to the package-level `Unmarshal`. Fields the payload does not mention are left at their zero value, which is the standard `encoding/json` behaviour and is why the next-to-last section of this chapter exists: if absence has to be an error, you have to ask for that.

A `jwt.Map` works too, and is right when the claim set is not known at compile time:

```
var m jwt.Map
err = verified.Claims(&m)
fmt.Printf("%T\n", m["iat"]) // json.Number
```

Numbers come back as `json.Number`, not `float64`, because the default `Unmarshal` decodes through `encoding/json/v2` with a custom unmarshal hook that reproduces what `json.Decoder.UseNumber()` gave `encoding/json` (v1): a number decoded into an `any` destination stays a string-backed `json.Number` rather than losing precision to `float64`. Call `.Int64()` on the result rather than asserting to `float64`.

Combining custom claims with standard ones

You have two ways to get registered claims into the same payload as your own.

Embed `jwt.Claims`. The fields are promoted and `encoding/json` inlines them, so one struct marshals to one flat object:

```
type userClaims struct {
	Username string `json:"username"`
	Role     string `json:"role"`
	jwt.Claims
```

```
Username: "kataras",
        Role:     "admin",
        Claims:   jwt.Claims{Subject: "user-42", Issuer: "auth.example.com"},
}, jwt.MaxAge(time.Hour))
```

```
{"username":"kataras","role":"admin","iss":"auth.example.com","sub":"user-
42","iat":1700000000,"exp":1700003600}
```

Pass them as sign options. That is the variadic fourth parameter of `Sign`, and a `SignOption` is one method:

```
func Sign(alg Alg, key PrivateKey, claims any, opts ...SignOption) ([]byte, error)

type SignOption interface {
        ApplyClaims(*Claims)
}

type SignOptionFunc func(*Claims)
```

`signToken` starts with a zero `Claims`, calls `ApplyClaims` on each option in the order you gave them, and merges the result with your claims. Because each option writes into the same struct, a later option overwrites an earlier one that sets the same field:

```
jwt.Sign(jwt.HS256, key, jwt.Map{}, jwt.Issuer("first"), jwt.Issuer("second"))
// payload: {"iss":"second"}
```

`Claims` itself implements `SignOption`, through an `ApplyClaims` that copies only the fields that are set:

```
func (c Claims) ApplyClaims(dest *Claims) {
        if v := c.NotBefore; v > 0 {
                dest.NotBefore = v
        }
        // ... and so on for the other seven fields
}
```

The test is `> 0` for the numbers and non-empty for the strings, so `jwt.Claims{}` as an option contributes nothing and a partially filled one contributes only what it names. That is what lets you keep a package-level template of your issuer and audience and pass it to every call.

One asymmetry follows. `ApplyClaims` will not write a zero, so a `Claims` option cannot clear a field an earlier option set. `NoMaxAge` exists precisely because of that: it is a `SignOptionFunc` assigning `0` to `Expiry` and `IssuedAt` directly, which no `Claims` value can do.

```
jwt.Sign(jwt.HS256, key, jwt.Map{"a": 1}, jwt.MaxAge(time.Hour), jwt.NoMaxAge)
// payload: {"a":1}
```

The sign-option shorthands

Before `signoptions.go`, the only sign options were `MaxAge`, `NoMaxAge`, `Audience` and `Claims`. Setting a `jti` or an `iss` on its own meant writing a `SignOptionFunc` by hand, and downstream packages did, which put this library's type into signatures that were meant to hide it. Seven shorthands now set one field of `Claims` each:

```
func ID(id string) SignOptionFunc
func OriginID(id string) SignOptionFunc
func Issuer(issuer string) SignOptionFunc
func Subject(subject string) SignOptionFunc
func NotBefore(t time.Time) SignOptionFunc
func IssuedAt(t time.Time) SignOptionFunc
func ExpiresAt(t time.Time) SignOptionFunc
```

The three taking a `time.Time` call `t.Round(time.Second).Unix()`, so a timestamp with nanoseconds in it lands on a whole second, which is what the format stores anyway.

Alongside them are the four originals. `MaxAge(d)` sets `exp` to now plus `d` and `iat` to now, both from the `Clock` variable. `NoMaxAge` clears both. `Audience` is the `[]string` type itself, which implements `ApplyClaims` so you can pass it directly. And `Claims`, as above. One behaviour of `MaxAge` is worth stating outright: if `maxAge <= time.Second` it returns `NoMaxAge`, so `jwt.MaxAge(500 * time.Millisecond)` produces a token with no expiry at all rather than one that expires immediately. Chapter 4 comes back to it.

Put together:

```
token, err := jwt.Sign(jwt.HS256, sharedKey, jwt.Map{"role": "admin"},
       jwt.MaxAge(15*time.Minute),
       jwt.Issuer("auth.example.com"),
       jwt.Subject("user-42"),
       jwt.ID("tok-1"),
       jwt.OriginID("tok-0"),
       jwt.Audience{"api.example.com"},
       jwt.NotBefore(time.Unix(1700000000, 0)),
)
```

```
{"role":"admin","nbf":1700000000,"iat":1700000000,"exp":1700000900,"jti":"tok-
1","origin_jti":"tok-0","iss":"auth.example.com","sub":"user-42","aud":
["api.example.com"]}
```

Your claims come first, then the accumulated standard ones in the field order of the `Claims` struct. That is not `encoding/json` being clever. It is the next section.

How Merge splices

`Merge` is what joins your claims to the standard ones, and it does not build a map and re-encode it:

```
func Merge(values ...any) ([]byte, error)
```

For each value it obtains JSON bytes, checks that the first byte is `{` and the last is `}`, strips both, and keeps the inside. At the end it joins the pieces with commas and wraps the lot in one pair of braces. Two objects go in and one comes out, with no intermediate `map[string]any` and no reparse. That is why it is fast, and it is also why this happens:

```
jwt.Merge(jwt.Map{"iss": "a"}, jwt.Claims{Issuer: "b"})
// {"iss":"a","iss":"b"}
```

The claim appears twice, because nothing looked at the names: looking at the names is the expensive part that was skipped on purpose.

Be clear about what that costs. RFC 8259 says the names in an object “should be unique” and that behaviour is unpredictable when they are not, so a payload like this is legal JSON but not what the specification prefers. `encoding/json` resolves it last-wins, so Go reads `b`, and so does anything else built on `encoding/json`, including `VerifiedToken.StandardClaims`:

```
token, _ := jwt.Sign(jwt.HS256, key, jwt.Map{"iss": "from-claims"}, jwt.Issuer("from-option"))
// payload:  {"iss":"from-claims","iss":"from-option"}
verified, _ := jwt.Verify(jwt.HS256, key, token)
// verified.StandardClaims.Issuer == "from-option"
```

A first-wins parser elsewhere reads `from-claims`, and both parsers are within their rights. If a token you issue is going to be read by something you did not write, do not put the same claim in two places. Set `iss` in your claims object or in a sign option, not in both.

`Merge` does check the two cases it cannot vouch for. A `string` or a `[]byte` is spliced in verbatim rather than marshalled, so those two are run through `json.Valid`:

```
_, err := jwt.Merge(`{"broken"}`)
// part: 1: jwt: value is not a JSON object
```

That is `ErrNotJSONObject`, which `Classify` reports as `KindMalformed`. The error names the part number rather than the value, because claims carry personal data and formatting the offending value into an error puts it in whatever you log. Anything that came out of `json.Marshal` skips the `json.Valid` scan, since it is valid by construction.

Nil values are skipped and empty objects are dropped, so `Merge(nil, jwt.Map{}, jwt.Map{"a": 1})` returns `{"a":1}` and merging nothing at all returns `{}`.

The Audience type

`aud` is the one registered claim whose type is ambiguous in the specification. RFC 7519 section 4.1.3 says it is an array of strings, then says that in the special case of one value it "MAY be a single case-sensitive string". Both forms are in the wild, so `Audience`, which is a `[]string`, gives itself an `UnmarshalJSON` that looks at the first byte and branches:

```
switch data[0] {
case '"': // it's a single string.
       // unmarshal into a string, wrap it in a one-element slice
case '[': // it's an array of strings.
       // unmarshal into []string directly
}
```

So `"aud":"api.example.com"` and `"aud":["api.example.com"]` both arrive as `Audience{"api.example.com"}`, and your code never has to care which the sender used. Anything else, a number or an object, falls through the switch and leaves the audience nil without an error. That is a quiet outcome for a malformed claim, and worth a length check or an `Expected` validator if `aud` matters to you.

Marshalling only goes one way. `jwt.Claims{Audience: jwt.Audience{"one"}}` writes `{"aud":["one"]}`, an array even for a single value, which is the form the specification prefers.

`Contains` is the membership test RFC 7519 section 4.1.3 actually asks for:

```
func (aud Audience) Contains(value string) bool

if !verified.StandardClaims.Audience.Contains("api.example.com") {
       return errors.New("token is for somebody else")
}
```

The rule in the specification is that the recipient must identify itself with a value in the audience. Not that it is the only value, and not that it is the first. A token good for `["api.example.com", "admin"]` is good for the API and good for the admin panel, and each of them checks for its own name. Comparison is exact and case sensitive, because audience values are identifiers rather

than prose: `Audience{"api.example.com"}.Contains("API.example.com")` is false. `Expected` calls `Contains` for you if you would rather express the check as a validator, and `String` joins the values with spaces for logging.

The second-chance parser

`verifyToken` unmarshals the payload into a `Claims` struct with the standard library. When that fails, it tries once more with a looser struct before giving up:

```
type claimsSecondChance struct {
	NotBefore json.Number `json:"nbf,omitempty"`
	IssuedAt  json.Number `json:"iat,omitempty"`
	Expiry    json.Number `json:"exp,omitempty"`
	ID        string      `json:"jti,omitempty"`
	OriginID  string      `json:"origin_jti,omitempty"`
	Issuer    any         `json:"iss,omitempty"`
	Subject   any         `json:"sub,omitempty"`
	Audience  Audience    `json:"aud,omitempty"`
}
```

`json.Number` for the timestamps and `any` for the two identity claims. `toClaims` then converts, and what it accepts and refuses is the point.

A float timestamp is accepted. Some authorities emit `"exp":2000000000.5`, off by a fraction of a second. `toUnix` parses through `Float64` and truncates, so that arrives as `2000000000`. This is the case the loose struct was written for.

A numeric identity claim is accepted and rendered as text. `{"sub":12345}` gives you `Subject` of `"12345"`. Issuers that write a numeric subject exist, and tolerating them is the other reason for the second pass.

A structured identity claim is refused. `{"sub":{"a":1}}` returns `ErrTokenForm`, as `jwt: invalid token form: sub: expected a string but got map[string]interface {}`. That is a fix rather than a feature: the conversion used to format the value with `%v`, which turned that payload into the subject `map[a:1]`, so an application treating `Subject` as a principal identifier accepted a token whose subject was not a principal at all. A boolean fails the same way.

A timestamp no `int64` can hold is refused. `{"exp":1e30}` returns `jwt: invalid token form: exp: out of range: 1e30`. This is the other repair, and it mattered more. `json.Number.Float64` reports an error for text `float64` cannot hold, and Go leaves a float-to-integer conversion undefined when the value does not fit. Both used to be discarded. An `exp` of `1e400` came out as a negative `int64`, failed the “greater than zero” test the old validation used, and was read as

no expiry at all. A token claiming an absurd expiry became a token that never expired. `toUnix` now checks `math.IsNaN`, `math.IsInf` and both `int64` bounds before converting.

When the second pass also fails to parse, verification reports an unexported error whose message is `jwt: payload is not a type of JSON`. `jwt.Plain` is the validator that suppresses it, for tokens whose payload is deliberately not JSON. `{"iat":"notanumber"}` lands here, since a string will not go into a `json.Number`.

Required fields

`encoding/json` has no notion of a required field. A claim that is absent leaves your struct field at its zero value, and a missing `username` is indistinguishable from an empty one.

The package adds an opt-in check driven by a `required` option in the JSON tag. Switch it on by replacing the package-level `Unmarshal`, which every `Claims` method routes through:

```
func UnmarshalWithRequired(payload []byte, dest any) error

jwt.Unmarshal = jwt.UnmarshalWithRequired

type strictClaims struct {
        Username string `json:"username,required"`
        Role     string `json:"role"`
}
```

It unmarshals normally and then walks the destination with reflection, descending into nested structs. Any exported field whose tag contains `,required` and whose value is zero produces `jwt: token is missing a required field: "Username"`. That is `ErrMissingKey`, which `Classify` reports as `KindRejected`. `HasRequiredJSONTag` exposes the tag test if you want to run it yourself over a `reflect.StructField`.

Two limits are worth stating. The check is on the Go value rather than the JSON, so a claim present but empty (`"username":""`) fails exactly as an absent one does. And the message names the Go field, `Username`, rather than the JSON name `username`, which is a nuisance when you are matching an error against a payload. Set `jwt.Unmarshal` once at startup, for the same reason as `MaxTokenSize`: it is a package variable read without a lock.

Summary

`Claims` holds the seven registered claims RFC 7519 defines, under Go names that differ from the wire names (`Expiry` is `exp`, `ID` is `jti`), plus `OriginID`, which serializes as `origin_jti`

and is this package's own convention for linking a refresh token to its access token. Every field carries `omitempty`, so zero means absent throughout.

Your own claims are any struct with JSON tags, read back with `VerifiedToken.Claims(&dest)`. Combine them with the registered ones by embedding `jwt.Claims`, or by passing sign options: `Claims` implements `SignOption` through an `ApplyClaims` that copies only non-zero fields, and `signoptions.go` adds `ID`, `OriginID`, `Issuer`, `Subject`, `NotBefore`, `IssuedAt` and `ExpiresAt` for setting one claim at a time. Later options win.

`Merge` joins the two objects by stripping braces and splicing raw JSON, which is why it is quick and why a claim set in two places appears twice in the payload. Go reads the last one, a first-wins parser somewhere else reads the first, so set each claim in exactly one place. Raw `string` and `[]byte` fragments are checked with `json.Valid` and rejected with `ErrNotJSONObject`.

`Audience` unmarshals from either a string or an array, always marshals as an array, and `Contains` is the RFC 7519 section 4.1.3 membership test. The second-chance parser tolerates a float timestamp and a numeric `sub`, returning `ErrTokenForm` for a structured `sub` or a timestamp no `int64` can hold, and `UnmarshalWithRequired` turns a `,required` tag into `ErrMissingKey`.

Further Reading

- RFC 7519 section 4.1: the registered claim names, one subsection each. Section 4.1.3 is the audience rule that `Contains` implements.
- IANA JSON Web Token Claims registry: the live list of registered names, and the place to check before inventing one.
- RFC 8259 section 4: JSON objects, and the paragraph on duplicate names that explains why `Merge` output is parser-dependent.
- encoding/json: struct tags, `omitempty`, embedded struct inlining, and the last-wins behaviour on duplicate keys.
- json.Number: what `UseNumber` gives you and why `.Int64()` beats a `float64` assertion on a timestamp.

CHAPTER 4

Expiry and Time

A bearer token is valid until it is not, and the only thing deciding when that moment arrives is a number in the payload. RFC 7519 makes every one of the time claims optional, which means the default state of a JWT is permanent. This chapter covers the three time claims, the exact rules this package applies to them, the two validators that exist because clocks disagree, and one function whose name points the opposite way from what it does.

The three time claims

RFC 7519 registers three claims that carry a time, all of them as seconds since the Unix epoch, and all three are listed as optional.

Claim	Name	Meaning	Section
`exp`	Expiration Time	When the token stops being valid.	4.1.4
`nbf`	Not Before	When the token starts being valid.	4.1.5
`iat`	Issued At	When the token was created.	4.1.6

In this package they are three fields on `Claims`:

```
type Claims struct {
        NotBefore int64 `json:"nbf,omitempty"`
        IssuedAt  int64 `json:"iat,omitempty"`
        Expiry    int64 `json:"exp,omitempty"`
        // ... iss, sub, aud, jti, origin_jti
}
```

`exp` is the one that matters for security. It puts a ceiling on the damage a leaked token can do, because a stolen bearer credential is the holder's credential until it expires and nothing about the token itself can tell you it was stolen.

`nbf` is for tokens minted ahead of the moment they are supposed to work: a subscription that starts at midnight, a batch job whose credentials are prepared in the previous run. Most systems never set it.

`iat` is metadata rather than a control. It tells you how old a token is and lets a verifier notice a clock that is badly wrong. It bounds nothing by itself.

`Verify` checks all three when they are present. There is no option to turn that off, and the check runs after the signature, so the values being checked are values your key covered.

Why zero means absent

Read `validateClaims` in `claims.go`. It is short, and every line is a decision:

```
func validateClaims(t time.Time, claims Claims) error {
	now := t.Round(time.Second).Unix()

	if claims.NotBefore != 0 {
		if now < claims.NotBefore {
			return ErrNotValidYet
		}
	}

	if claims.IssuedAt != 0 {
		if now < claims.IssuedAt {
			return ErrIssuedInTheFuture
		}
	}

	if claims.Expiry != 0 {
		if now > claims.Expiry {
			return ErrExpired
		}
	}

	return nil
}
```

The three fields are `int64` tagged `omitempty`. That tag makes a `Claims` value with no expiry serialize to a payload with no `exp` key, and it also makes zero unrepresentable as a real value: nothing in the round trip distinguishes “the issuer did not set this” from “the issuer set this to 1970-01-01T00:00:00Z”. So zero is read as absent and the claim is skipped.

Every other value is present and gets checked, negative ones included. A token carrying `"exp": -1` expired one second before the epoch, and `now > claims.Expiry` is true for any clock set after 1969, so `Verify` returns `ErrExpired`.

Both comparisons are strict, and the reference time is rounded to the nearest second. A token is therefore accepted from the exact second named by its `nbf`, which is what RFC 7519 asks for, and still accepted during the exact second named by its `exp`, which is a second more generous

than a strict reading of “the current time MUST be before `exp`”. The RFC allows small leeway of this kind. In practice it means every window is a whole number of seconds wide, and sub-second precision does not exist anywhere in this format.

That sounds obvious and it was not always the code. The test used to be `claims.Expiry > 0`, which reads as “if an expiry was set” and behaves as “if the expiry is after 1970”. Under that rule a negative `exp` counted as no expiry at all, so a token that should have been rejected outright verified and stayed valid forever. `toUnix` reaches the same place from another direction: it converts a JSON number to `int64`, and a float too large for `int64` used to land on the most negative value the type can hold, so an `"exp"` of `1e400` produced an immortal token. Both are fixed, and `toUnix` now returns an error wrapped in `ErrTokenForm` for anything out of range.

One asymmetry to know about. `Claims.ApplyClaims`, which runs when you pass a `Claims` value as a `SignOption`, copies each timing field only `if v > 0`. A template `Claims` carrying a negative `Expiry` is therefore ignored on the way in, while a token arriving with a negative `exp` is treated as expired on the way out.

Setting a lifetime with MaxAge

You will rarely fill `Expiry` in by hand:

```
token, err := jwt.Sign(jwt.HS256, key, claims, jwt.MaxAge(15*time.Minute))
```

`MaxAge` sets two claims, not one:

```
func MaxAge(maxAge time.Duration) SignOptionFunc {
	if maxAge <= time.Second {
		return NoMaxAge
	}

	return func(c *Claims) {
		now := Clock()
		c.Expiry = now.Add(maxAge).Unix()
		c.IssuedAt = now.Unix()
	}
}
```

`iat` comes along with `exp` because the two are read together. An `exp` with no `iat` tells a reader when the token dies but not how long it was meant to live, which is what `Claims.Age()` reports.

Now the guard at the top. Any duration of one second or less returns `NoMaxAge`, and `NoMaxAge` is not a no-op:

```
var NoMaxAge SignOptionFunc = func(c *Claims) {
        c.Expiry = 0
        c.IssuedAt = 0
}
```

It zeroes both fields, `omitempty` drops them, and the token goes out with no `exp` and no `iat`. State that plainly, because it is the sentence people need: **a zero `MaxAge` in a configuration file produces permanent tokens, and no call in the chain returns an error.** `Sign` succeeds. `Verify` succeeds, tomorrow and next year. Nothing logs anything.

The route into that is short. A `KeyConfiguration` carries a `MaxAge time.Duration` field with `json`, `yaml`, `toml` and `ini` tags, and `Keys.SignToken` applies it only under `if k.MaxAge > 0`. Leave `MaxAge` out of the YAML, or write `MaxAge: 0`, and the option is never added. Write `MaxAge: 1s` and the option is added and then erases itself. Two different mistakes, one result.

If your claims are a `Map` rather than a struct, the equivalent is `func MaxAgeMap(maxAge time.Duration, claims Map)`, which mutates the map in place and returns nothing:

```
claims := jwt.Map{"user_id": "12345", "role": "admin"}
jwt.MaxAgeMap(15*time.Minute, claims)

token, err := jwt.Sign(jwt.HS256, key, claims)
```

Two differences from `MaxAge`. It will not overwrite an `exp` already in the map, so a caller who set one keeps it. And for a duration of one second or less it returns without touching the map rather than zeroing anything, so the failure here is “no expiry was added” rather than “an expiry was removed”. Both end at a token with no `exp`.

Requiring an expiry

RFC 7519 leaves `exp` optional and this package follows the RFC. If you want the stricter rule, ask for it:

```
verifiedToken, err := jwt.Verify(jwt.HS256, key, token, jwt.RequireExpiry)
```

`RequireExpiry` is a `TokenValidatorFunc` value, not a function returning one, so it takes no arguments. Its body does three things in order: return any error that arrived from earlier validation, return `ErrMissingExpiry` if `standardClaims.Expiry == 0`, otherwise return nil. That first step matters. A token that has genuinely expired already failed with `ErrExpired`, which this validator passes through, so it only ever sees the missing case. The error message is `jwt: token has no expiry`.

Add it wherever you accept tokens you did not mint, and to your own issuer's verification too, because that is what catches the configuration mistake above on the day it ships rather than on the day somebody audits the key store.

There is a second reason, about revocation. A blocklist entry has to be kept until the token would have expired anyway, at which point it can be dropped. A token with no expiry has no such point, so its revocation record can never be collected. Chapter 9 goes into it.

Clock skew and the order of validators

Two machines rarely agree on the time to the second. An issuer whose clock runs two minutes fast mints a token whose `iat` is in the future from the verifier's point of view, and `validateClaims` rejects it with `ErrIssuedInTheFuture` even though nothing is wrong with it. There are two validators for this, and the difference between them is not obvious from the names.

```
func Future(dur time.Duration) TokenValidatorFunc
func Skew(d time.Duration) TokenValidatorFunc
```

`Future` opens with `if errors.Is(err, ErrIssuedInTheFuture)` and does nothing for any other error. It rescues `iat`, and only `iat`. Now look again at `validateClaims`: it checks `nbf` first and returns as soon as that check fails. An issuer that sets `nbf` equal to `iat`, which several do, produces a skewed token that fails with `ErrNotValidYet`, and `Future` passes that error straight through. The rescue never happens.

This is not hypothetical. Four call sites across two services depended on `Future` and worked only because the identity provider they talked to happens to omit `nbf`. A change at the provider would have broken all four at once.

`Skew(d)` is the one to reach for:

```
verifiedToken, err := jwt.Verify(jwt.HS256, key, token, jwt.Skew(2*time.Minute))
```

It switches on the error it was handed and accepts either `ErrNotValidYet` or `ErrIssuedInTheFuture` when the claim falls within `d` of now. A non-positive `d` passes the error through unchanged, so `Skew(0)` is a safe no-op rather than a validator that accepts everything.

Tolerance in the other direction is deliberately not offered. There is no option to accept a token that expired thirty seconds ago. Extending the life of a credential that has run out is a policy

decision about your system, not a correction for a clock, and if you want it you should write it where a reviewer can see it.

Validator order is load-bearing. `Verify` runs validators in the order you pass them, handing each one the error the previous produced, and it breaks out of the loop at the first validator that returns non-nil. A validator that rescues an error must therefore come before one that would refuse it. Almost every built-in validator, `Expected` among them, opens with `if err != nil { return err }`, which is the correct way to respect an earlier failure and also the reason ordering matters:

```
// Correct: Skew clears the skew error, then Expected checks the issuer.
jwt.Verify(alg, key, token, jwt.Skew(2*time.Minute), jwt.Expected{Issuer: "us"})

// Wrong: Expected returns ErrNotValidYet, Verify breaks, Skew never runs.
jwt.Verify(alg, key, token, jwt.Expected{Issuer: "us"}, jwt.Skew(2*time.Minute))
```

If you keep validators in a slice and want to append at a call site, `Validators` saves you the conversion loop and preserves order:

```
jwt.Verify(alg, key, token, jwt.Validators(app.Validators, jwt.Skew(time.Minute))...)
```

One consequence of the early return in `validateClaims` deserves a sentence. When `Skew` clears an `nbf` failure, the `iat` and `exp` checks for that token never ran, because `validateClaims` returned before reaching them. In practice that only matters for a token whose `exp` is earlier than its `nbf`, which nobody meant to issue, but it is worth knowing that a cleared error is a skipped check rather than a passed one.

Leeway is stricter than it sounds

`Leeway` reads like a tolerance. It is the opposite.

```
func Leeway(leeway time.Duration) TokenValidatorFunc
```

The whole of its logic is one comparison, guarded by `err == nil` and `Expiry > 0`:

```
if Clock().Add(leeway).Round(time.Second).Unix() > standardClaims.Expiry {
        return ErrExpired
}
```

It moves the clock forward by `leeway` and then asks whether the token has expired. So `jwt.Leeway(30 * time.Second)` rejects, with `ErrExpired`, any token that will expire within the next thirty seconds. It narrows the acceptance window instead of widening it.

The name comes from the caller's problem rather than the token's. You are about to start work that will take a while: a database transaction, a chain of calls to three services, a batch that runs for twenty seconds. Beginning that with a credential that dies halfway through leaves you with a partial write and a 401 you cannot retry. `Leeway` is how you insist on a token with enough life left to finish. Note the two guards: it does nothing at all for a token with no `exp`, and it never rescues an earlier error, only refuses.

Replacing the clock in tests

Every time comparison in this package goes through one package-level variable:

```
var Clock = time.Now
```

`validateClaims`, `MaxAge`, `MaxAgeMap`, `Leeway`, `Future`, `Skew` and `Claims.Timeleft` all call it. Replace it and you control the package's idea of now, which is how you test expiry without sleeping:

```
func TestExpiredToken(t *testing.T) {
	original := jwt.Clock
	t.Cleanup(func() { jwt.Clock = original })

	base := time.Date(2024, 1, 1, 12, 0, 0, 0, time.UTC)
	jwt.Clock = func() time.Time { return base }

	token, err := jwt.Sign(jwt.HS256, key, jwt.Map{"sub": "user"},
jwt.MaxAge(time.Hour))
	if err != nil {
		t.Fatal(err)
	}

	jwt.Clock = func() time.Time { return base.Add(2 * time.Hour) } // past the
expiry

	if _, err = jwt.Verify(jwt.HS256, key, token); !errors.Is(err, jwt.ErrExpired)
{
		t.Fatalf("expected ErrExpired but got: %v", err)
	}
}
```

`t.Cleanup` is doing real work there. A test that leaves `Clock` pointing at a fixed instant breaks every later test that expects real time, and the failure surfaces somewhere else entirely.

Set it once, at startup. `Clock` is read without a lock, from whichever goroutine is verifying, so assigning to it while requests are in flight is a data race and the race detector will say so. That rules out swapping the clock in a test that runs in parallel with others: keep those serial, or build

the token you need from explicit `Claims` values and leave the global alone. The same applies to `CompareHeader`, `ReadFile`, `Marshal`, `Unmarshal` and `MaxTokenSize`.

Reading time off a verified token

Three methods on `Claims` answer the questions you will actually ask:

```
func (c Claims) Age() time.Duration
func (c Claims) ExpiresAt() time.Time
func (c Claims) Timeleft() time.Duration
```

`Age` is `exp` minus `iat`: the lifetime the issuer intended, not the time elapsed since issuance. It does not consult the clock. With either claim missing it returns whatever the subtraction produces, so check for zero before you trust it.

`ExpiresAt` converts `exp` to a `time.Time` at second precision. With no `exp` you get `time.Unix(0, 0)`, the epoch, which is emphatically not "no expiry" if you feed it to a comparison.

`Timeleft` is `exp` minus `Clock()`, so it is the one that moves. Positive means the token is still good, negative means it has expired, and zero means either that it expires exactly now or that there is no `exp` at all. Those last two are indistinguishable in the return value, which is a good reason to pair it with `RequireExpiry`:

```
verifiedToken, err := jwt.Verify(jwt.HS256, key, token, jwt.RequireExpiry)
if err != nil {
	return err
}

claims := verifiedToken.StandardClaims
if left := claims.Timeleft(); left < 2*time.Minute {
	log.Printf("token for %s expires at %s, %s left",
		claims.Subject, claims.ExpiresAt().Format(time.RFC3339), left)
}
```

`StandardClaims` is a field on `*VerifiedToken`, already parsed. Your own claims come out separately through `verifiedToken.Claims(&dest)`.

Choosing lifetimes

There is no correct number, but there is a shape, and it comes from one fact: you cannot un-issue a JWT. Verification is a signature check and a clock comparison, neither of which asks

anything else, which is the whole reason a JWT scales and the reason a token stays good until it expires whatever happens in between.

Access tokens: minutes. Five to fifteen is the usual range. This is the token that travels on every request, gets logged by proxies, sits in browser memory, and shows up in crash reports. A short lifetime gives a leak a deadline. The cost is refresh traffic and the client-side machinery to handle a 401 mid-flight without losing the user's work.

Refresh tokens: days to weeks. Seven days is a common default. This token is presented rarely, to one endpoint, and can be stored somewhere better guarded. The cost is that it becomes the thing worth stealing, which is why it should be revocable: keep its `jti` and check a blocklist when it is presented. Chapter 9 covers that, and the pairing is common enough that the package ships a `TokenPair` type for it.

Anything you cannot revoke: as short as you can stand. For a token minted for one operation, a machine-to-machine call or a signed download link, the lifetime should cover the operation and no more.

Permanent tokens: almost never. If you need a credential that does not expire you also need a way to revoke it, which means a lookup on every request, which means the JWT is buying you nothing over an opaque random string in a database. Use the database.

One more number to set deliberately: the tolerance you pass to `Skew` should be smaller than your shortest token lifetime, or you have quietly extended it. On hosts running NTP, a minute or two is generous, and reaching for ten usually means a clock is broken somewhere and the tolerance is hiding it.

Summary

`exp`, `nbf` and `iat` are all optional in RFC 7519, so a token with none of them is a valid token that never expires. `Verify` checks all three when they are present, after the signature. Zero means absent, because `int64` plus `omitempty` leaves nothing else to mean it. Every other value is checked, negative ones included, so `exp: -1` is expired rather than unbounded.

`MaxAge(d)` sets both `exp` and `iat`, and for any `d` of one second or less it returns `NoMaxAge`, which clears both. A zero `MaxAge` in a config file therefore yields permanent tokens silently. `RequireExpiry` is the check that catches it, and it returns `ErrMissingExpiry`.

For clock skew use `Skew(d)`, which covers `nbf` and `iat`. `Future(d)` covers only `iat`, and since `validateClaims` returns at the first failed check it cannot rescue a token whose issuer set `nbf`. Validators run in the order given and `Verify` stops at the first error, so a rescuing

validator must come first. `Leeway(d)` is stricter rather than looser: it rejects tokens expiring within the next `d`.

`Clock` is a package variable, ideal for tests, read without a lock, so set it before anything concurrent starts. `Age()`, `ExpiresAt()` and `Timeleft()` read the claims you have already verified.

Further Reading

- RFC 7519 section 4.1.4: the `exp` claim, including the sentence that makes it optional and the note about leeway between clocks. Sections 4.1.5 and 4.1.6 cover `nbf` and `iat`.
- RFC 8725: JSON Web Token Best Current Practices: the operational advice RFC 7519 leaves out, including why a claim you did not validate is not a claim you know.
- pkg.go.dev/github.com/kataras/jwt: the generated reference for `MaxAge`, `Leeway`, `Skew`, `Future` and `RequireExpiry`.
- pkg.go.dev/time#Duration: what `2*time.Minute` is, and why a `time.Duration` written as a bare integer is nanoseconds.

CHAPTER 5

Choosing an Algorithm

The `alg` argument you pass to `Sign` and `Verify` decides more than which mathematics runs. It decides who can mint tokens, how large every token is, whether the same claims produce the same bytes twice, and what your key material looks like on disk. This chapter goes through the six families the package implements, states what each one requires and costs, and ends with a recommendation you can disagree with on purpose.

The six families

Every algorithm in the package is a package-level variable of type `Alg`, and the interface has three methods:

```
type Alg interface {
	Name() string
	Sign(key PrivateKey, headerAndPayload []byte) ([]byte, error)
	Verify(key PublicKey, headerAndPayload []byte, signature []byte) error
}
```

`PrivateKey` and `PublicKey` are both aliases for `any`. That is what lets one interface carry a `[]byte` secret and an `*rsa.PrivateKey`, and it is also why a key of the wrong type is a runtime error rather than a compile error. The next section deals with that.

Family	Names	Sign key	Verify key	RFC
HMAC-SHA2	`HS256` `HS384` `HS512`	`[]byte`	the same `[]byte`	7518 3.2
RSA PKCS#1 v1.5	`RS256` `RS384` `RS512`	`*rsa.PrivateKey`	`*rsa.PublicKey`	7518 3.3
ECDSA	`ES256` `ES384` `ES512`	`*ecdsa.PrivateKey`	`*ecdsa.PublicKey`	7518 3.4
RSA-PSS	`PS256` `PS384` `PS512`	`*rsa.PrivateKey`	`*rsa.PublicKey`	7518 3.5
Unsecured	`NONE`	ignored	ignored	7518 3.6
EdDSA	`EdDSA`	`ed25519.PrivateKey`	`ed25519.PublicKey`	8037

The number in an HMAC, RSA or PSS name is the SHA-2 variant and nothing else. `RS512` is not a bigger RSA key than `RS256`: both use whatever key you hand them and both produce a signature the width of the modulus. ECDSA is the exception, because the curve and the hash move together:

```
ES256 Alg = &algECDSA{"ES256", crypto.SHA256, 32, 256}
ES384 Alg = &algECDSA{"ES384", crypto.SHA384, 48, 384}
ES512 Alg = &algECDSA{"ES512", crypto.SHA512, 66, 521}
```

The last two fields are the coordinate width in bytes and the curve size in bits. Note the final line: `ES512` uses P-521, not a 521-typo and not a 512-bit curve. The name follows the hash, the curve does not, and 66 bytes is what 521 bits rounds up to.

RSA-PSS is the same RSA key as PKCS#1 v1.5, with different padding:

```
PS256 Alg = &algRSAPSS{"PS256", &rsa.PSSOptions{SaltLength: rsa.PSSSaltLengthAuto,
Hash: crypto.SHA256}}
```

A `*rsa.PrivateKey` works with both families. What you cannot do is verify a `PS256` token with `RS256` or the reverse: the padding differs, so the signature does not check out, and the header comparison would refuse it first anyway.

Symmetric or asymmetric

This is the decision. Everything else in the chapter is detail.

With HMAC, the key that verifies a token is the key that signs one. There is no separate public half. Any service you give the verification key to can mint tokens indistinguishable from yours, and your `Verify` call will accept them, because they are correct.

Say that plainly, because it is easy to walk past. If you run an auth service and six APIs, and all seven share an HS256 secret, then all seven are your auth service. A bug in the least careful of them, a leaked environment variable in a container image, a contractor with read access to one repository: any of those is a signing key. Nothing in the token can tell you afterwards which of the seven produced it, because the signature proves possession of a key that all seven possess.

With RSA, ECDSA or EdDSA the split is real. The issuer holds the private key and mints tokens; everybody else holds the public key and can only check them. That public key can be published, put in a config map, served from a JWKS endpoint, or checked into a repository.

The trade is honest in both directions. Asymmetric verification costs more CPU and produces larger tokens, and you now have a key pair to generate, distribute and rotate, which is Chapters

6 and 7. HMAC is faster, the key is a byte slice you can generate with one call, and there is nothing to parse. The question is not which is stronger. It is whether everything that verifies your tokens is allowed to issue them.

HMAC is a reasonable choice inside one trust boundary. One process signing and verifying its own session tokens, or two services under the same deployment with the same owner, is a fine place for HS256. The moment a third party verifies, or a verifier lives somewhere you would not put a signing key, you want the split. And note the thing HMAC cannot do at all: every holder of the key is equally the author, so if you ever have to answer "which service issued this token", the token itself will not tell you.

Key types and wrong keys

Each `Sign` and `Verify` begins with a type assertion, and a wrong type yields `ErrInvalidKey`.

```
func (a *algHMAC) Sign(key PrivateKey, headerAndPayload []byte) ([]byte, error) {
	secret, ok := key.([]byte)
	if !ok {
		return nil, fmt.Errorf("expected a string: %w", ErrInvalidKey)
	}
	// ...
}
```

The most common mistake in the whole package is right there. `HS256` wants `[]byte`, and a Go string is not a `[]byte`:

```
_, err := jwt.Sign(jwt.HS256, "my-secret", claims)         // ErrInvalidKey
_, err = jwt.Sign(jwt.HS256, []byte("my-secret"), claims)  // fine
```

The error text reads "expected a string", which is misleading, since a string is exactly what it will not take. Match on the wrapped error rather than the message: `errors.Is(err, jwt.ErrInvalidKey)`.

The asymmetric implementations follow the same shape with one addition. RSA, ECDSA and EdDSA all check for a nil key after the assertion:

```
publicKey, ok := key.(*rsa.PublicKey)
if !ok || publicKey == nil {
	if privateKey, ok := key.(*rsa.PrivateKey); ok && privateKey != nil {
		publicKey = &privateKey.PublicKey
	} else {
		return ErrInvalidKey
	}
}
```

The nil test is not decoration. A nil pointer stored in an interface makes the interface itself non-nil, so an `*rsa.PublicKey(nil)` passes the assertion and then panics inside `crypto/rsa`. It got there through a registered `Key` whose `Public` field was a typed nil, and from that point any token naming that `kid` crashed the process. EdDSA has the matching version: `ed25519.PrivateKey.Public()` slices the key at `[32:]`, so the length check has to run before the call rather than after.

The fallback in that snippet is deliberate. All three asymmetric families accept a private key where a public one is expected and extract the public half themselves, which is a convenience for the single-process case where you hold both. It does not work in reverse. There is no way to sign with a public key.

Two more refusals worth knowing. `algECDSA.Sign` compares the key's curve against the algorithm's `curveBits` and returns `ErrInvalidKey` when they differ, so signing with `ES256` and a P-384 key fails outright rather than producing something a verifier rejects later. And `algECDSA.Verify` checks `len(signature) != 2*a.keySize` before doing any arithmetic, because the JWS form is the fixed-width `r || s` concatenation rather than a DER structure.

Sizes and cost

Signature width is fixed by the algorithm, so token size is predictable. These were measured by signing the same three-claim payload with each algorithm, using a freshly generated 2048-bit RSA key:

Algorithm	Signature	Whole token
`none`	0 bytes	105 chars
`HS256`	32 bytes	149 chars
`HS384`	48 bytes	170 chars
`HS512`	64 bytes	192 chars
`ES256`	64 bytes	192 chars
`EdDSA`	64 bytes	192 chars
`ES384`	96 bytes	234 chars
`ES512`	132 bytes	282 chars
`RS256` and `PS256`	256 bytes	448 chars

RSA is the outlier, and the reason is structural: an RSA signature is the width of the modulus, so a 2048-bit key gives 256 bytes whatever the hash, and a 4096-bit key gives 512. An RS256

token is roughly three times the size of an ES256 one carrying identical claims. For a token that rides in a header on every request that is real bandwidth, and for anything that has to fit in a 4 KB cookie it can be the deciding constraint.

For cost, the package's benchmark suite covers HS256 only. On windows/arm64:

```
BenchmarkSign_Map-12       489426     2500 ns/op    1528 B/op    23 allocs/op
BenchmarkSign_Struct-12    580483     2179 ns/op    1392 B/op    21 allocs/op
BenchmarkVerify-12         431294     2620 ns/op    1376 B/op    18 allocs/op
```

Almost all of that is JSON and base64, not HMAC. The same suite measures `golang-jwt` at 4713 ns/op and `go-jose` at 7460 ns/op for the equivalent verification, which tells you about the packages rather than about SHA-256.

Per-algorithm numbers are not in the suite, so measure them on your own hardware rather than trusting a table in a book. What you can rely on is the shape, which follows from the mathematics rather than the implementation. HMAC is the cheapest at both ends by a wide margin, because it is one hash. RSA is lopsided: signing performs a private-key operation across the full modulus and is the slowest operation in the set, while verification uses the public exponent, usually 65537, and is among the fastest. ECDSA is the reverse, cheap to sign and dearer to verify. EdDSA is quick at both and needs no entropy at signing time.

Pick on the ratio your system actually has. A gateway verifying ten thousand tokens a second against an issuer minting fifty cares about verification cost; a batch job minting a million tokens cares about the other end.

Determinism and golden tests

Sign the same header and payload twice with the same key. Do you get the same token? It depends on the family, and the answer was measured rather than assumed:

Family	Same bytes twice?
HMAC	Yes
RSA PKCS#1 v1.5	Yes
EdDSA (Ed25519)	Yes
RSA-PSS	No
ECDSA	No

HMAC is a hash of the input. PKCS#1 v1.5 padding is fixed, so `rsa.SignPKCS1v15` is deterministic even though it takes an `io.Reader`. Ed25519 derives its nonce from the message

and the private key by design, which is one of the reasons the scheme exists: there is no per-signature random value to get wrong. PSS and ECDSA both draw from `rand.Reader` on every call, PSS to salt the padding and ECDSA to pick a nonce, and in both cases repeating that value would leak the private key. The randomness is not optional and not a flaw.

The consequence lands in your test suite. A golden test that pins an expected token string works for HS256, RS256 and EdDSA, and cannot work for ES256 or PS256:

```
// Fine for HS256, RS256, EdDSA.
if string(token) != want {
	t.Fatalf("token changed:\n got %s\nwant %s", token, want)
}

// Required for ES256, PS256: verify instead of comparing.
if _, err := jwt.Verify(jwt.ES256, publicKey, token); err != nil {
	t.Fatal(err)
}
```

The package's own golden tests take exactly this split: where they can pin bytes they pin bytes, which is what caught the header casing bug in the next section, and where they cannot they verify and check the claims. If you are choosing partly for testability, the deterministic algorithms are easier to assert on, which is a genuine if minor point in favour of EdDSA over ECDSA.

The none algorithm

RFC 7518 section 3.6 defines an unsecured JWT: the `alg` header reads `none`, the signature segment is empty, and nothing is proved. The package implements it in three short methods, of which this is the interesting one:

```
func (a *algNONE) Name() string {
	return "none"
}
```

Lowercase, because that is what the RFC says. It returned `"NONE"` until v0.2.0, which no other implementation recognises, so tokens produced here were not valid unsecured JWTs anywhere else and conforming tokens naming `none` were never accepted here. Both directions were broken, which is a small argument for byte-exact golden tests. `Sign` returns `nil, nil`, and `Verify` returns `ErrTokenSignature` unless the signature is empty.

Now the part that matters. The classic JWT attack is to take a signed token, rewrite its header to `{"alg":"none"}`, drop the signature, and hand it back. It works against any library that reads the algorithm out of the token. This one does not:

```
verifiedToken, err := jwt.Verify(jwt.HS256, key, token)
```

The algorithm is an argument. The default `CompareHeader` builds the expected header from that argument and compares bytes:

```
expectedHeader := createHeaderRaw(alg)
if !bytes.Equal(expectedHeader, headerDecoded) {
	return nil, nil, nil, ErrTokenAlg
}
```

`createHeaderRaw("HS256")` is the 27-byte literal `{"alg":"HS256","typ":"JWT"}`. There is no parse and no lookup. A header reading `{"alg":"none","typ":"JWT"}` is 26 bytes that cannot equal those 27, so it fails with `ErrTokenAlg` before any signature check runs. The comparison also accepts the reversed field order some libraries emit, and a 15-byte header with no `typ`, but each of those is likewise built from the algorithm you pinned. **A `Verify` call that named a real algorithm can never accept an `alg: none` token.**

`Enrich` refuses `NONE` outright, both in the package-level function and in `UnverifiedToken.Enrich`:

```
if alg == NONE {
	return nil, fmt.Errorf("%w: refusing to enrich under the unsecured algorithm",
ErrTokenAlg)
}
```

Enriching means adding claims and signing the result. Doing that under `none` produces a token with your claims, your structure and no signature: the shape of a credential without being one. There is no legitimate use, so it is refused rather than documented.

If you genuinely want an unsecured token, for a client-side preferences blob where tampering costs nothing, pass `jwt.NONE` to `Sign` and `Verify` explicitly and be certain no authorization decision reads it. The source file listing the algorithms marks `NONE`, and also `HS256`, `HS384` and `HS512`, with the comment `// Not recommended.`

Writing your own Alg

`Alg` is exported, so you can implement it. Reasons that come up: an HSM or KMS that holds the private key and exposes a signing call, an algorithm the package does not ship, or a wrapper around one that already works. The wrapper is the easiest case, because embedding `jwt.Alg` supplies the two methods you are not changing:

```
// timed wraps an algorithm and reports how long each signing operation took.
type timed struct {
	jwt.Alg
	observe func(name string, d time.Duration)
}

func (a timed) Sign(key jwt.PrivateKey, headerAndPayload []byte) ([]byte, error) {
	start := time.Now()
	defer func() { a.observe(a.Alg.Name(), time.Since(start)) }()

	return a.Alg.Sign(key, headerAndPayload)
}
```

Pass it as you would any algorithm: `jwt.Sign(timed{Alg: jwt.ES256, observe: record}, key, claims)`. `Name` and `Verify` come from the embedded value, so the header still reads `ES256` and the token verifies with plain `jwt.ES256` anywhere else. A KMS-backed implementation is the same shape with a real `Sign` body, and it has to return the JWS signature form, which for ES256 is the 64-byte `r || s` concatenation rather than the DER structure most signing APIs hand back.

Three things to get right.

The name goes into the header, and the header is built by concatenation. Nothing escapes it, so `Name()` is validated before it gets there. `encodeToken` opens with `checkAlgName`, which requires a name of 1 to 64 characters drawn only from `a–z`, `A–Z`, `0–9` and the four punctuation marks `-`, `+`, `.` and `_`, returning a wrapped `ErrTokenAlg` otherwise. That is the character set JWA uses for algorithm names, not an allowlist of the built-in ones, so a sensible custom name still works. Without the check, an implementation returning a name containing a double quote could inject header fields of its own choosing into a token signed with somebody else's key. Your `Name()` should return a constant.

Verification must be constant time in whatever you compare. If you compare bytes yourself, use `hmac.Equal` or `crypto/subtle`, not `bytes.Equal`.

Implement `AlgParser` if your keys come from files or a registry. It is the optional second interface, and `Keys` uses it to turn PEM bytes into key objects:

```
type AlgParser interface {
	Parse(private, public []byte) (PrivateKey, PublicKey, error)
}
```

All the built-in asymmetric algorithms implement it. `HS256` does not, because there is nothing to parse.

One limit to note: `UnverifiedToken.Alg()` resolves a name through `parseAlg`, which only searches the built-in list, so it returns `ErrTokenAlg` for a custom name. Signing and verification are unaffected, since you pass the `Alg` value directly. Code that inspects an unverified token to decide what to do with it is affected, and the fix is a custom `HeaderValidator`. Chapter 6 covers those.

What to pick

EdDSA, when you control both ends. 64-byte signatures, 32-byte public keys, fast at both signing and verification, deterministic, and no per-signature randomness to get wrong. It is the newest of the set, which is the whole of its downside: older JWT libraries may not support it, and HSM coverage is thinner than for RSA and ECDSA. If your issuer and your verifiers are all your own Go services, that downside does not apply to you.

RS256 or ES256, when you must interoperate. RS256 is the most widely implemented asymmetric JWT algorithm in existence; every library, every gateway, every identity provider handles it. Choose it when an unknown third party has to verify, or when you are consuming tokens from a provider that offers it. ES256 gives you the same interoperability almost everywhere with a token a third of the size, so prefer it when you can check that your consumers support it. Both are fine choices, and the difference between them is bandwidth rather than strength.

HMAC only inside one trust boundary. HS256 is fast, simple, and requires a key at least 32 bytes of real entropy, which `jwt.MustGenerateRandom(32)` will give you. Use it when the same owner runs everything that touches the key, and stop using it the moment that stops being true. A password, a config default, or a phrase somebody typed is not a key; RFC 7518 requires a key at least as long as the hash output for a reason.

PS256 over RS256 when both ends are modern. PSS has better proofs and no known weaknesses in its padding, where PKCS#1 v1.5 has a long history of implementation problems. The catch is support: PSS is less widely implemented, and OpenSSL uses distinct key OIDs to stop a key being reused across the two schemes. Pick PSS for a new closed system, and RS256 when interoperability decides.

ES384, ES512, HS512 and the rest: only if something requires them. Larger hashes and curves cost bytes and cycles and buy margin above what the rest of your system provides. If a compliance document names one, use it. Otherwise the 256-bit variants are not the weak link.
NONE: no.

Whatever you choose, pin it: pass the algorithm to `Verify` rather than letting a header choose. And write the choice down with its reason, because the next person to touch it will otherwise

change it to whatever the first search result recommends.

Summary

Six families: HMAC (`HS256`, `HS384`, `HS512`), RSA PKCS#1 v1.5 (`RS*`), RSA-PSS (`PS*`), ECDSA (`ES256`, `ES384`, `ES512`), EdDSA (Ed25519), and `NONE`. The number names the hash, except in ECDSA where it moves with the curve, and `ES512` uses P-521.

The real decision is symmetric or asymmetric. With HMAC every verifier can also mint tokens that your own `Verify` will accept, and nothing in the token says which holder produced it.

Key types are checked by assertion at the top of every `Sign` and `Verify`, and a wrong type returns `ErrInvalidKey`. A Go string for an HMAC key is the common case. The asymmetric implementations also check for typed nils, because a nil pointer in an interface asserts successfully and then panics.

Signatures are 32 to 64 bytes for HMAC, 64 to 132 for ECDSA, 64 for Ed25519, and the width of the modulus for RSA, which is 256 bytes at 2048 bits. RSA signing is the slowest operation in the set and RSA verification among the fastest. HMAC, RSA PKCS#1 v1.5 and Ed25519 produce identical bytes for identical input; RSA-PSS and ECDSA randomise, so golden tests that pin a token string cannot be written for them.

`NONE` names itself `none`, per RFC 7518 section 3.6. Because header comparison is byte-exact against the algorithm you pinned, an `alg: none` token can never be accepted by a `Verify` call naming a real algorithm, and `Enrich` refuses `NONE` outright.

Recommendation: EdDSA where you control both ends, RS256 or ES256 where you must interoperate, HMAC only inside one trust boundary.

Further Reading

- RFC 7518: JSON Web Algorithms: sections 3.2 to 3.6 define every algorithm here, including the minimum HMAC key lengths and the `none` algorithm.
- RFC 8037: how Ed25519 is used in JOSE, which is where the name `EdDSA` comes from.
- RFC 8032: Ed25519 itself, including why its signatures are deterministic.
- RFC 8725 section 3.1: perform algorithm verification, which is the practice this package enforces by making `alg` an argument. Section 3.2 covers choosing among them.
- NIST SP 800-57 Part 1: the key-length equivalence tables, if you want to argue about 2048-bit RSA against P-256 with numbers.

- pkg.go.dev/crypto/ed25519 and pkg.go.dev/crypto/ecdsa: the Go implementations these algorithms call, including the key types you have to supply.

CHAPTER 6

Keys and Rotation

A system with one signing key has no way to replace it. Every token in flight was signed with that key, every verifier holds that key, and the moment you swap it every outstanding token becomes invalid at once. This chapter is about the mechanism that removes the constraint: a key identifier in the header, a registry mapping identifiers to keys and algorithms, and the ordering that lets a new key take over while the old one is still verifying traffic.

The flag day problem

Chapters 2 and 3 passed the key as an argument: `jwt.Sign(jwt.HS256, secret, claims)` on one side, `jwt.Verify(jwt.HS256, secret, token)` on the other. That works until you need a different key, and then there are two ways to replace it and both are bad. Deploy the new key everywhere at the same instant, and every process that verifies must restart in lockstep with every process that signs, while any token issued a moment before the switch fails on the other side. Or accept a window where verification is broken and hope it is short.

That coordinated restart is a flag day. It scales with the number of services holding the key, it cannot be rehearsed safely, and it turns key replacement into an event rather than a routine. The predictable result is that keys never get replaced, so a leaked key has no recovery path except the flag day you have been avoiding. What breaks the deadlock is letting a verifier hold several keys, and letting the token say which one signed it.

The kid header and the Keys registry

`kid` is a header parameter registered by RFC 7515 section 4.1.4. It is a hint, not a credential: it names a key, and the receiver decides what that name means. The value is an arbitrary case-sensitive string, so dates, version counters and random identifiers all work. What matters is that it stays stable for the life of a key and is never reused for another. The registry is a map, and each entry is a `Key`:

```
type Keys map[string]*Key

type Key struct {
        ID      string        // Unique identifier for this key
        Alg     Alg           // Algorithm implementation for this key
        Public  PublicKey     // Public key for verification (can be nil for HMAC)
        Private PrivateKey    // Private key for signing (required for signing)
```

```
        MaxAge  time.Duration // Optional: automatic token expiration
        Encrypt InjectFunc    // Optional: payload encryption function
        Decrypt InjectFunc    // Optional: payload decryption function
}
```

Field	What it holds
`ID`	The `kid` written into the header when this key signs, and the map key it is found under. `Register` keeps the two in agreement.
`Alg`	An `Alg` value such as `jwt.RS256`, not a string. The most important field here: it binds one algorithm to this key, on your side rather than the sender's.
`Public`, `Private`	The key material. Both types are aliases for `any`, so the concrete type is whatever the algorithm expects: `*rsa.PrivateKey` for `RS256`, `*ecdsa.PrivateKey` for `ES256`, `ed25519.PrivateKey` for `EdDSA`, `[]byte` for the HMAC family. A verify-only registry leaves `Private` nil.
`MaxAge`	When greater than zero, adds an expiry to every token this key signs, which saves repeating `jwt.MaxAge(...)` at each call site.
`Encrypt`, `Decrypt`	A payload cipher, covered in Chapter 11. Setting either one makes this key unusable for enrichment and for configuration export, both of which need the plain payload.

Registering takes four arguments, and `Get` reads a key back:

```
func (keys Keys) Register(alg Alg, kid string, pubKey PublicKey, privKey PrivateKey)
func (keys Keys) Get(kid string) (*Key, bool)

keys := make(jwt.Keys)
keys.Register(jwt.RS256, "signing-2024-01", &rsaPrivate.PublicKey, rsaPrivate)
keys.Register(jwt.ES256, "signing-2024-06", &ecdsaPrivate.PublicKey, ecdsaPrivate)
```

`Register` overwrites an existing entry with the same identifier without complaint. That is deliberate, and it also means a copy-pasted line with an unedited `kid` silently discards a key, surfacing later as `ErrUnknownKid` on tokens you expected to verify. `Get` returns a pointer into the map, so mutating the result mutates the registry, which is the supported way to set `MaxAge`, `Encrypt` or `Decrypt` after `Register`.

Signing and verifying through the registry

Signing names the key. The header it produces carries both parameters, `kid` first, and an unregistered identifier returns `ErrUnknownKid` before any crypto runs:

```
func (keys Keys) SignToken(kid string, claims any, opts ...SignOption) ([]byte, error)

token, err := keys.SignToken("signing-2024-06", userClaims{Sub: "u-1", Role: "admin"},
       jwt.MaxAge(15*time.Minute))
// header: {"kid":"signing-2024-06","alg":"ES256"}
```

On MaxAge precedence. `SignToken` prepends the key's `MaxAge` to the options you passed, and options apply in order, so the last one wins. A key registered with a one hour `MaxAge` and signed with `jwt.MaxAge(time.Minute)` produces a token expiring in one minute. The method's own doc comment claims the opposite. What is written here is what the code does, measured by reading `exp` off the signed token.

Verification does not name a key. It reads one out of the header. `VerifyToken` unmarshals into your struct and hands back only an error; `Verify` returns the token itself, for when you need the raw bytes to forward, a claim outside the standard set, or `StandardClaims` to give to a blocklist:

```
func (keys Keys) VerifyToken(token []byte, claimsPtr any, validators
...TokenValidator) error
func (keys Keys) Verify(token []byte, validators ...TokenValidator) (*VerifiedToken,
error)

verifiedToken, err := keys.Verify(token)
if err != nil {
       return err
}
var claims userClaims
err = verifiedToken.Claims(&claims)
```

Both run the same path. `Verify` exists because without it every caller who wanted the token hand-wrote `jwt.VerifyWithHeaderValidator(nil, nil, token, keys.ValidateHeader, validators...)`, including the two leading nils, which are the algorithm and key arguments the header validator makes redundant.

Three errors are worth knowing by name: `ErrEmptyKid` when the header carries no `kid`, `ErrUnknownKid` when it carries one you have not registered, and `ErrTokenAlg` when the header's algorithm disagrees with the registered one. All three are decided before any signature work happens.

ValidateHeader binds the kid to an algorithm

Everything above rests on one method, which satisfies the `HeaderValidator` function type. Given a decoded header it parses `kid` and `alg`, looks the key up, and returns the algorithm, the public key and the decryption function to use. A key registered with no algorithm returns `ErrInvalidKey` rather than dereferencing a nil interface. The line that matters is the last one here:

```
func (keys Keys) ValidateHeader(alg string, headerDecoded []byte) (Alg, PublicKey,
InjectFunc, error)

if h.Alg != key.Alg.Name() {
        return nil, nil, nil, ErrTokenAlg
}
```

The algorithm comes from the registry, and the header's `alg` is only permitted to agree with it. Consider what that prevents. An attacker takes a token whose `kid` is `signing-2024-06`, a key you registered as `ES256`, rewrites the header to say `RS256`, and signs with an RSA key of their own. A naive implementation reads `RS256` from the header, fetches the key material for that `kid`, and attempts a verification it was never meant to attempt. Here the header is rejected with `jwt: unexpected token algorithm` before the signature is examined. The same check covers the downgrade where `alg` is replaced with `none`, and the RS256-to-HS256 confusion where a published public key is submitted as an HMAC secret.

This is the security property of the whole chapter, and it fits in one line: **the token chooses which key, and the registry chooses which algorithm**. The sender picks an entry from a list you wrote, and everything about that entry comes from your side. The `alg` parameter is a further constraint: when it is not empty the header must match it as well, which pins verification to a single algorithm across a registry holding several.

HMAC in a registry

HMAC signs and verifies with the same bytes, so an `HS256` key has no separate public half. `Register` accepts nil there and fills it in:

```
keys.Register(jwt.HS256, "internal", nil, []byte("sercrethatmaycontainch@r$"))

// inside Register:
if pubKey == nil {
```

CHAPTER 7

JWKS

Chapter 6 gave every key a name and put that name in the header. This chapter is about how the names and the keys travel between machines. A JSON Web Key Set is the format almost every identity provider uses to publish its public keys over HTTP, and this package can produce one, consume one, and keep a consumed one current while the process runs. It also refuses several things when fetching, and each refusal exists because of a specific way the naive version fails.

What a key set is

RFC 7517 defines two things. A JWK is a JSON object describing one cryptographic key. A JWKS is a JSON object with a single member, `keys`, holding an array of them. That is the whole container:

```
{
  "keys": [
    {
      "kty": "RSA",
      "kid": "signing-2024-01",
      "use": "sig",
      "alg": "RS256",
      "n": "uGCsN_H_19DmAEz-
OokW7VzP07cZHp41zfShi37N80IPhwaT_JSGzerqjVxt0p2lerBIJg2BPFtOLTQ",
      "e": "AQAB"
    },
    {
      "kty": "EC",
      "kid": "signing-2024-06",
      "use": "sig",
      "alg": "ES256",
      "crv": "P-256",
      "y": "P9N3oC-gFahsyubWhsH-ELO3c6KLh8UCVDIY8NtR6SI",
      "x": "7z0-SC3-OWY_b0ihYuayoh67zt70l-tXsyZKUGIQoho"
    }
  ]
}
```

The Go type is `JWK`. Four members appear on every key and the rest depend on the key type:

Member	Applies to	Meaning
`kty`	all	Key type: `RSA`, `EC` or `OKP`. It decides which of the members below are present.
`kid`	all	The identifier a token's `kid` header names. This is the link back to Chapter 6.
`use`	all	`sig` for signature keys, `enc` for encryption. This package always writes `sig`.
`alg`	all	The algorithm the key is for: `RS256`, `ES256`, `EdDSA` and so on.
`crv`	EC, OKP	Curve name: `P-256`, `P-384`, `P-521` or `Ed25519`.
`n`, `e`	RSA	Modulus and public exponent, base64url encoded. `e` is almost always `AQAB`, which is 65537.
`x`, `y`	EC	The coordinates of the public point, base64url encoded.
`x`	OKP	The 32-byte Ed25519 public key itself. There is no `y`.

Members that do not apply to a key type are omitted, not emitted empty. That is what the `omitempty` tags on `Crv`, `N`, `E`, `Y` and `X` are for. An earlier version published EC keys with empty `n` and `e` and RSA keys with an empty `crv`, which RFC 7517 does not permit and which some consumers reject outright. The four common members carry no `omitempty`, so they are always present. The values are base64url without padding, the same encoding Chapter 1 covered, applied to the big-endian bytes of each number.

Nothing in a JWKS is secret: these are public keys, and their purpose is to be downloadable by anyone who needs to check a signature. What matters is integrity, not confidentiality, which is why the transport rules later in this chapter are strict.

Publishing your own keys

If you issue tokens that other services verify, publish a key set and let them fetch it. The alternative is emailing PEM files around, which works exactly once and then becomes the reason nobody rotates.

One method converts a registry into a key set. Only the public halves are read, so a registry holding private keys is safe to pass. The convention is to serve the result at `/.well-known/jwks.json`, and to build the body once at startup rather than per request, because the document changes only when a key is added or retired:

```
func (keys Keys) JWKS() (*JWKS, error)

func jwksHandler(keys jwt.Keys) (http.HandlerFunc, error) {
	set, err := keys.JWKS()
	if err != nil {
		return nil, err
	}

	body, err := json.Marshal(set)
	if err != nil {
		return nil, err
	}

	sum := sha256.Sum256(body)
	etag := `"` + hex.EncodeToString(sum[:8]) + `"`

	return func(w http.ResponseWriter, r *http.Request) {
		w.Header().Set("Content-Type", "application/jwk-set+json")
		w.Header().Set("ETag", etag)
		w.Header().Set("Cache-Control", "public, max-age=3600")

		if r.Header.Get("If-None-Match") == etag {
			w.WriteHeader(http.StatusNotModified)
			return
		}

		w.Write(body)
	}, nil
}
```

RFC 7517 registers `application/jwk-set+json` for this document. Most providers send `application/json` instead, and no consumer this package talks to checks either way, so send the registered one.

On the ordering. `Keys` is a map, and ranging over a map in Go yields a different order each time, so `JWKS()` sorts the result by `kid` with `slices.SortFunc` and `strings.Compare` before returning it. Without that sort the same registry produced a different byte sequence on every call. Any ETag computed from the body would then change on every request, every conditional request would miss, and a document that genuinely changes twice a year would be re-downloaded by every client on every poll. Sorting makes the body a pure function of the registry, which is what makes the handler above work.

One limit carried over from Chapter 6: a registry containing an HMAC key fails here with `unsupported public key type: []uint8`, because a shared secret has no public half to publish. Keep symmetric keys in a separate registry from the one you serve.

Generating a single JWK

The per-key conversion is exported, for when you are assembling a set by hand or publishing one key from a store this package does not manage:

```
func GenerateJWK(kid string, alg string, publicKey PublicKey) (*JWK, error)

jwk, err := jwt.GenerateJWK("2024-06", "ES256", &privateKey.PublicKey)
// {"kty":"EC","kid":"2024-06","use":"sig","alg":"ES256","crv":"P-
256","y":"LnZi...","x":"quCA..."}
```

It accepts `*rsa.PublicKey`, `*ecdsa.PublicKey`, `ecdsa.PublicKey` and `ed25519.PublicKey`, and returns `unsupported public key type: %T` for anything else. The `alg` argument is passed through untouched for RSA and ECDSA, so it is on you to pass one that matches the key: nothing stops you labelling a P-256 key `ES512`, and a consumer that trusts the label will then reject every token it signs. Ed25519 ignores the argument and always writes `EdDSA`, because there is nothing else it could be.

Fetching somebody else's keys

Two functions read a key set over HTTP:

```
func FetchJWKS(client HTTPClient, rawURL string) (*JWKS, error)
func FetchPublicKeys(url string) (Keys, error)
```

`FetchPublicKeys` is the whole job in one call: fetch, parse, convert. It returns a `Keys`, the same registry type from Chapter 6, so everything that worked there works here:

```
keys, err := jwt.FetchPublicKeys("https://auth.example.com/.well-known/jwks.json")
if err != nil {
        return err
}

var claims userClaims
err = keys.VerifyToken(token, &claims)
```

`FetchJWKS` is the lower half, returning the parsed `*JWKS` before conversion. Use it when you want the raw set, or when you need control over the request. Its first argument is an interface

with one method, which `*http.Client` satisfies, and so does a wrapper of your own that adds an `Authorization` header for a private endpoint, records a metric, or returns a canned response in a test. Passing nil uses the package's own client, described in the next section.

```
type HTTPClient interface {
	Get(string) (*http.Response, error)
}

func (set *JWKS) PublicKeys() Keys
```

`PublicKeys` is the conversion step. It walks the array, resolves each `alg` against the built-in algorithms, converts the key material, and builds a `Key` with the `kid` as its identifier. Keys it cannot handle are skipped silently: an unrecognised `alg`, an unsupported `kty`, base64url that does not decode. That is deliberate, so a set containing one key type you do not support still yields the ones you do. It is also why a set this package understands nothing in comes back as an empty map with no error at all, and `KeySet` below turns exactly that case into an error.

What the fetch refuses to do

The fetching path is small and most of it is refusals. Each one replaced a default that was wrong in a specific way.

A 15 second timeout. The default client used to be `http.DefaultClient`, which has no timeout at all. An endpoint that accepted the connection and then said nothing held the calling goroutine for the lifetime of the process. If you pass your own `HTTPClient`, set a timeout on it yourself, because the package cannot reach inside your implementation to add one.

HTTPS, except on loopback. A key set fetched over plain HTTP can be replaced in transit by anyone on the path, and the replacement keys will then verify the attacker's tokens, which turns the whole verification step into theatre. Loopback is allowed because that is where a local identity provider runs while you develop, and the exemption is exactly four hostnames: `localhost`, `127.0.0.1`, `::1` and `[::1]`.

```
_, err := jwt.FetchJWKS(nil, "http://auth.example.com/.well-known/jwks.json")
// jwt: jwks: refusing to fetch keys over "http": use https
```

At most five redirects, each re-checked. `http.Client` follows up to ten redirects by default and does not care where they go, including from HTTPS to plain HTTP. The package's client caps at five and runs the same scheme check on every hop, so a provider cannot be redirected off a secure origin partway through.

One megabyte, on both reads. `MaxJWKSSize` is `1 << 20`. The response length is chosen by the host being fetched from, which on this path is precisely the host whose keys you have not established yet, so an unbounded `io.ReadAll` hands a stranger control of your memory. The limit wraps the body before both the error read and the JSON decode. A key set of a few dozen keys is a handful of kilobytes, so a megabyte leaves room to spare.

```
body := io.LimitReader(resp.Body, MaxJWKSSize)
if resp.StatusCode >= 400 {
	b, _ := io.ReadAll(body) // a truncated body is still worth reporting.
	return nil, &HTTPError{StatusCode: resp.StatusCode, Body: b}
}
```

The error body is on a field, not in the message. A status of 400 or above returns an `*HTTPError`, whose `Error()` is `jwt: jwks: status code: 404` and nothing more:

```
type HTTPError struct {
	StatusCode int
	Body       []byte
}

var httpErr *jwt.HTTPError
if errors.As(err, &httpErr) && httpErr.StatusCode == http.StatusNotFound {
	return errors.New("no key set at that address")
}
```

Keeping those bytes out of the message is the point. An error string almost always ends up in a log, and the remote host chooses every byte of that body, newlines and terminal escape sequences included. Putting it in the message hands a stranger a log line to write. Read the field yourself when you want the detail, and treat it as untrusted text when you do. The type was unexported until recently, which meant callers could not tell a 404 from a 503 even though the package's own documentation showed them doing it.

KeySet keeps the keys current

`FetchPublicKeys` is one GET. It has no cache, no expiry, and no way to fetch again when a token names a key it has not seen. Every service verifying against somebody else's keys has to build that part, and two that were examined did not: one called `FetchPublicKeys` inside a constructor, so the whole fleet failed with `ErrUnknownKid` until restarted whenever the issuer rotated, and the other pasted the provider's public key into a configuration file as PEM. `KeySet` is that missing piece:

```
func NewRemoteKeySet(url string, options ...RemoteKeySetOption) (*KeySet, error)

keySet, err := jwt.NewRemoteKeySet("https://auth.example.com/.well-known/jwks.json",
       jwt.WithRefreshInterval(15*time.Minute),
       jwt.WithMinRefreshInterval(time.Minute),
)
if err != nil {
       return err
}
defer keySet.Close()

verifiedToken, err := keySet.Verify(token, jwt.Expected{Issuer:
"https://auth.example.com/"})
```

The first fetch happens before the constructor returns, so a URL that is wrong, unreachable or serving nothing usable fails at startup rather than on the first request. Three options adjust it:

Option	Default	Effect
`WithRefreshInterval(d)`	1 hour	How often the background goroutine refetches. Zero disables the timer entirely, which is reasonable for a provider that rotates rarely.
`WithMinRefreshInterval(d)`	1 minute	The shortest gap between two fetches triggered by an unknown key.
`WithHTTPClient(c)`	the 15 second client above	The client used for every fetch.

The methods are few. `Keys` returns the current map, which you must not modify. `Refresh` fetches now regardless of the timers. `ValidateHeader` is the same interface `Keys` implements in Chapter 6, so a `KeySet` drops into `jwt.VerifyWithHeaderValidator` anywhere a registry did, and `Verify` is that call with the nils already filled in. `Close` stops the background goroutine, is safe to call more than once, and leaves the keys readable afterwards.

```
func (ks *KeySet) Keys() Keys
func (ks *KeySet) Refresh() error
func (ks *KeySet) ValidateHeader(alg string, headerDecoded []byte) (Alg, PublicKey,
InjectFunc, error)
func (ks *KeySet) Verify(token []byte, validators ...TokenValidator) (*VerifiedToken,
error)
func (ks *KeySet) Close() error
```

Two triggers, and why the second is rate limited. The timer is the ordinary one. The interesting trigger is in `ValidateHeader`: when the current map has no key for the token's `kid`, it refetches

once and tries again. That is what makes somebody else's rotation invisible to you. The provider publishes the new key before signing with it, and the first token carrying the new `kid` causes the refetch that closes the gap, with no restart and no configuration change.

That second trigger is an outbound request caused by an inbound one, which is a shape worth being careful with. A token naming a key that will never exist is a request anybody can send, so without a floor a stream of them becomes a stream of requests to your identity provider, from your address, at whatever rate the sender chooses. `WithMinRefreshInterval` is that floor, and the slot is claimed before the fetch rather than after, so a burst of goroutines all hitting an unknown `kid` produces one request between them rather than one each.

Reads take no lock. The map is swapped whole through an `atomic.Pointer`, and a verification already in flight keeps reading the map it started with. That is the reason this is a separate type instead of locking inside `Keys`: a mutex there would cost every verification in every program to serve the callers who rotate at runtime.

Failures are quiet on purpose. A failed refresh leaves the previous keys in place, because a provider having a bad five minutes should not take your verification down with it, and the keys you already hold are almost certainly still correct. The background goroutine swallows both the error and any panic, since nothing is recovering above it, so if you want to know about refresh failures, call `Refresh` yourself on your own schedule and handle its error.

`ErrKeySetEmpty`. `PublicKeys` skips keys it cannot convert, so an endpoint serving a set this package does not understand yields an empty map and no error. `Refresh` treats a zero-length result as a failed refresh and returns `ErrKeySetEmpty`, which `NewRemoteKeySet` surfaces at startup. Without it the operator sees `unknown kid` on every request and goes looking for a token problem, when the actual fault is that the key fetch is returning nothing usable.

AWS Cognito

Cognito publishes a key set per user pool, at a URL built from the region and the pool identifier. Fetching it once is one call:

```
func FetchAWSCognitoPublicKeys(region, userPoolID string) (Keys, error)
```

It returns an `AWSCognitoError` when Cognito answers with a status of 400 or above and a JSON body this package can parse, carrying the status and the message Cognito sent. That message is quoted and prefixed, `jwt: aws cognito: status code: 400: "User pool does not exist."`, for the same reason `HTTPError` keeps its body off the message: the text comes from a remote host and ends up in your logs. Like `FetchPublicKeys`, it fetches once and never again, so a pool that rotates its signing key leaves the result stale until you call again.

The character allowlist. Both arguments are interpolated into the URL that is then fetched, so both are checked first. A region of `evil.com/x` would move the request to a host of the caller's choosing, and whatever came back would be used to verify tokens. The check is an allowlist rather than an escape, because neither field has any business containing something that needs escaping: letters, digits and hyphens, plus underscore for the pool identifier, since Cognito writes those as `us-east-1_AbC123`:

```
_, _, err := jwt.NewCognitoKeySet("us east 1", "us-east-1_AbC123", appClientID)
// jwt: aws cognito: region contains an unexpected character ' '
```

For anything longer-lived than a script, use the `KeySet` constructor instead. Its middle return value is the part that matters: an `Expected` validator holding the issuer, `https://cognito-idp.{region}.amazonaws.com/{userPoolId}`, and the audience, which is your app client identifier.

```
func NewCognitoKeySet(region, userPoolID, appClientID string, options
...RemoteKeySetOption) (*KeySet, Expected, error)

keySet, expected, err := jwt.NewCognitoKeySet("us-east-1", "us-east-1_AbC123",
appClientID)
if err != nil {
	return err
}
defer keySet.Close()

verifiedToken, err := jwt.VerifyWithHeaderValidator(nil, nil, token,
	keySet.ValidateHeader, expected, jwt.Skew(2*time.Minute))
```

Why the signature and the clock are not enough. A Cognito user pool can have many app clients, and every one of them gets tokens signed by the same pool key. A token minted for a different app client of the same pool therefore has a valid signature under a key in your key set, an issuer you recognise, and an expiry in the future. If you check only the signature and the timestamps, you accept it, and whatever that other client is allowed to do, its users can now do to you. Both services examined for this chapter verified the signature and the clock and nothing else, which is why the constructor hands you the checks rather than leaving them as an exercise. `Skew` is in the example because Cognito omits `nbf` and its clock can run slightly ahead of yours; Chapter 4 covers what that allowance costs. If you pass an empty `appClientID`, the returned `Expected` carries only the issuer, and you are back to accepting tokens for any client in the pool.

Summary

A JWKS is a JSON object with one member, `keys`, holding JWK objects. Each JWK carries `kty`, `kid`, `use` and `alg`, plus the members its key type needs: `n` and `e` for RSA, `crv`, `x` and `y` for EC, `crv` and `x` for Ed25519. Members that do not apply are omitted rather than emitted empty.

`Keys.JWKS()` builds a set from a registry, reading only public halves and sorting by `kid` so the serialized body is stable between calls, which is what makes ETag caching work. Serve it at `/.well-known/jwks.json`, built once at startup. `GenerateJWK` does one key at a time. `FetchPublicKeys` fetches, parses and converts in one call, returning the same `Keys` type Chapter 6 used; `FetchJWKS` is the half below it and takes an `HTTPClient`; `JWKS.PublicKeys()` skips anything it cannot convert rather than failing the whole set.

The fetch enforces a 15 second timeout, HTTPS outside loopback, five redirects each re-checked, and a one megabyte cap on both the error body and the decode. `HTTPError` carries the response body on a field rather than in the message, because a remote host chooses those bytes and error strings end up in logs.

`KeySet` keeps a fetched set current. It refreshes on a timer and again when a token names an unknown `kid`, with the second trigger rate limited because anybody can send such a token. Reads are lock-free, failed refreshes keep the previous keys, and an empty result is `ErrKeySetEmpty`. `NewCognitoKeySet` returns the key set together with the issuer and audience checks, because a Cognito token minted for a different app client of the same pool has a valid signature under a key you hold.

Further Reading

- RFC 7517: JSON Web Key and JSON Web Key Set. Section 4 is the member list; appendix A has worked examples of each key type.
- RFC 7518 section 6: how the key material is encoded per key type, including why `e` is normally `AQAB`.
- RFC 8037: the `OKP` key type and the `Ed25519` curve.
- RFC 7638: JWK thumbprints, a way to derive a `kid` from the key material so two parties compute the same one.
- RFC 9110 section 8.8.3: ETag, and what a strong validator means for the handler above.
- Verifying a JSON Web Token (AWS): Amazon's own list of the checks a Cognito token needs, including the audience check.

- pkg.go.dev/github.com/kataras/jwt: the generated reference for every signature quoted here.

CHAPTER 8

Validators

`Verify` checks a signature and three timestamps. Every other question your application has about a token, which issuer minted it, which audience it names, whether somebody logged out two minutes ago, arrives through one interface with one method. This chapter covers that interface, the validators the package ships, and one property of the loop that runs them: it stops at the first error. That decides the order your validators have to be in, and getting it backwards produces a chain that reads correctly and never runs.

The interface and its function form

Two declarations, both in `verify.go`:

```
type TokenValidator interface {
	ValidateToken(token []byte, standardClaims Claims, err error) error
}

type TokenValidatorFunc func(token []byte, standardClaims Claims, err error) error

func (fn TokenValidatorFunc) ValidateToken(token []byte, standardClaims Claims, err
error) error
```

`token` is the whole token exactly as it arrived: header, payload and signature, dots included, not the payload and not decoded. `standardClaims` is the parsed `Claims` value, the seven registered claims plus `origin_jti`, and nothing else from the payload reaches you here.

`err` is the third parameter and the one that makes this interface unusual. It is the verdict so far: whatever claims validation produced, or whatever the previous validator returned. A validator receives the state of the decision and returns the new one, which is what lets it clear an error rather than only add one.

`TokenValidatorFunc` is the closure form. `Leeway`, `Future` and `Skew` each return one, `RequireExpiry` and `Plain` are one, and `Expected` and `*Blocklist` are types carrying the method. All go in the same variadic slot of `Verify`, and none ever runs for a token whose signature failed, since `decodeToken` returns early on any signature or format problem.

The chaining rule

The whole mechanism is the last third of `verifyToken` in `verify.go`:

```
if err == nil { // do not proceed if we have a JSON error.
        err = validateClaims(Clock(), standardClaims)
}
for _, validator := range validators {
        // A token validator can skip the builtin validation and return a nil error.
        if err = validator.ValidateToken(token, standardClaims, err); err != nil {
                break
        }
}
```

Read the assignments rather than the shape. Going into the loop, `err` holds one of three things: nil, `errPayloadNotJSON` if the payload would not parse, or a timing error from `validateClaims`. Note the guard on that call: when the payload is not JSON the timing checks never run at all, which matters two sections from here. Inside the loop, `err` is both the value passed in and the variable assigned, and whatever comes out is what `Verify` returns. So trace the first iteration. Validator one returns non-nil, the loop breaks, nothing else runs. Validator one returns nil, so validator two is handed nil. There is no third path.

A validator only ever receives a non-nil error when it is first in the list. Stated as a program rather than an argument, over a token that expired an hour ago:

```
spy := func(name string) jwt.TokenValidatorFunc {
        return func(_ []byte, _ jwt.Claims, err error) error {
                fmt.Printf("%s saw err=%v\n", name, err)
                return err
        }
}
jwt.Verify(jwt.HS256, key, expired, spy("first"), spy("second"), spy("third"))
// prints exactly one line: first saw err=jwt: token expired
```

The second and third spies print nothing. On a token that passes, all three run and all three see nil. Three consequences follow.

A rescuing validator has to come first. `Skew`, `Future` and `Plain` work by turning one specific error into nil. Anywhere but position one they are handed nil, and returning nil for nil changes nothing:

```
// Correct: Skew clears the nbf failure, then Expected checks the subject.
jwt.Verify(jwt.HS256, key, token, jwt.Skew(2*time.Minute), jwt.Expected{Subject:
"u1"}) // <nil>
// Wrong: Expected passes ErrNotValidYet through, the loop breaks, Skew never runs.
jwt.Verify(jwt.HS256, key, token, jwt.Expected{Subject: "u1"},
jwt.Skew(2*time.Minute))
// -> jwt: token not valid yet
```

Both compile and both look like a validator chain, and the second silently rejects every token from an issuer whose clock is a minute fast.

A later validator cannot repair an earlier failure. Once any validator returns an error verification is over, and nothing downstream can overturn that decision.

The prologue every built-in validator opens with is unreachable except in position one. `Expected`, `RequireExpiry` and `*Blocklist` all start with `if err != nil { return err }`. Write it in your own anyway: it is correct where it runs, and whoever reorders the chain next year will not read your code first.

The package's own documentation got this wrong until v0.2.0. The doc comment on `Plain` said to place it last, "so it can catch JSON errors that other validators might depend on". Run `jwt.Verify(jwt.HS256, key, plain, jwt.Expected{Issuer: "us"}, jwt.Plain)` over a non-JSON payload and the result is `jwt: payload is not a type of JSON`: `Expected` returned the error it was handed, and `Plain` never executed. The comment now says the opposite, which is the useful reminder that advice about ordering is worth testing rather than trusting.

Expected, field by field

`Expected` is a distinct type over the same struct as `Claims`, so you fill it the same way:

```
type Expected Claims

func (e Expected) ValidateToken(token []byte, c Claims, err error) error
```

Each field is checked only when it is set, which is what makes partial validation possible, and the error names the claim:

Field	Claim	Checked when	Error
`NotBefore`	`nbf`	`> 0`	`jwt: field not match: nbf`
`IssuedAt`	`iat`	`> 0`	`jwt: field not match: iat`
`Expiry`	`exp`	`> 0`	`jwt: field not match: exp`
`ID`	`jti`	non-empty	`jwt: field not match: jti`
`Issuer`	`iss`	non-empty	`jwt: field not match: iss`
`Subject`	`sub`	non-empty	`jwt: field not match: sub`
`Audience`	`aud`	non-empty	`jwt: field not match: aud ("admin")`

All of them wrap `ErrExpected`, so `errors.Is(err, jwt.ErrExpected)` matches and `Classify` returns `KindRejected`. `OriginID` has no check: the field exists because `Expected` is `Claims`, and `ValidateToken` never reads it. The timestamp fields are exact equality against a Unix second, useful in a test over a token you minted and rarely what a handler wants.

The audience row is the one entry in that table that is not equality. Each audience you ask for has to appear somewhere in the token's list, which is what RFC 7519 section 4.1.3 asks of a recipient, so neither order nor extra audiences matter:

```
for _, expected := range e.Audience {
	if !c.Audience.Contains(expected) {
		return fmt.Errorf("%w: aud (%q)", ErrExpected, expected)
	}
}
// Token carries aud: ["api", "web", "mobile"].
jwt.Verify(jwt.HS256, key, token, jwt.Expected{Audience: jwt.Audience{"web"}})
// <nil>
jwt.Verify(jwt.HS256, key, token, jwt.Expected{Audience: jwt.Audience{"mobile",
"api"}}) // <nil>
jwt.Verify(jwt.HS256, key, token, jwt.Expected{Audience: jwt.Audience{"admin"}})
// -> jwt: field not match: aud ("admin")
```

This used to compare the two lists for equal length and equal order, so an identity provider adding a second audience broke every recipient at once. What the membership test does not do

is tell you the token was meant only for you: `["api", "partner-api"]` passes a check for `api`, the RFC's intent. If your rule is stricter, compare `StandardClaims.Audience` yourself.

A zero-value `Expected` validates nothing, and the call site cannot tell. The line `jwt.Verify(jwt.HS256, key, token, jwt.Expected{})` returns nil for any token that verifies. It has a validator in it, it passes review, and it checks no claim. The usual route to it is a config struct whose `Issuer` was never populated, so check that the string is non-empty when you load your configuration. The validator will not complain on your behalf.

Plain, for a payload that is not JSON

Nothing in RFC 7515 requires a JWS payload to be JSON. This package assumes it is, and when the assumption fails it sets an unexported error before the validator loop, so the only way to act on it is with a validator:

```
var errPayloadNotJSON = errors.New("jwt: payload is not a type of JSON")

var Plain = TokenValidatorFunc(func(token []byte, standardClaims Claims, err error)
error {
	if err == errPayloadNotJSON {
		return nil // skip this error entirely.
	}
	return err
})
```

Pass it first and the payload comes back raw: for a token signed over `just some text`, `jwt.Verify(jwt.HS256, key, plain, jwt.Plain)` returns nil and `verifiedToken.Payload` is that text.

Understand what you gave up. `validateClaims` is guarded by `if err == nil` and runs before the loop, so when the payload does not parse the timing checks never happened, and `Plain` clearing the error afterwards does not send the verifier back to run them. A token verified with `Plain` has a checked signature and no time bound at all, which also makes `Plain` followed by any claim-based validator pointless. `Plain` then `Expected{Issuer: "us"}` returns `jwt: field not match: iss`, and `Plain` then `RequireExpiry` returns `jwt: token has no expiry`, both from the zero `Claims`. `errPayloadNotJSON` is not a name `Classify` knows, so it comes back as `KindInvalid` rather than `KindMalformed`.

Reach for `Plain` when you verify tokens some other system minted with a non-JSON payload, and not to make a parse failure go away in your own tokens, where it is the finding.

Leeway, Skew, Future and RequireExpiry

Four validators that exist because of clocks and defaults. Chapter 4 works through the timing rules; here is what the chaining rule adds to them.

```
func Leeway(leeway time.Duration) TokenValidatorFunc
func Skew(d time.Duration) TokenValidatorFunc
func Future(dur time.Duration) TokenValidatorFunc
var RequireExpiry = TokenValidatorFunc(...)
```

`Leeway` reads like a tolerance and behaves as the opposite. Its whole body is one comparison, which moves the clock forward and then asks whether the token has expired:

```
if Clock().Add(leeway).Round(time.Second).Unix() > standardClaims.Expiry {
        return ErrExpired
}
```

A token minted with `MaxAge(30*time.Second)` verifies on its own and returns `jwt: token expired` under `jwt.Leeway(time.Minute)`. The name comes from the caller's problem rather than the token's: you are about to start something slow, and beginning it with a credential that dies halfway through leaves a partial write and a 401 you cannot retry. Two guards. It does nothing for a token with no `exp`, and it is wrapped in `if err == nil`, so it only refuses.

`Skew(d)` rescues both `ErrNotValidYet` and `ErrIssuedInTheFuture` when the claim falls within `d` of now. `Future(dur)` rescues only `ErrIssuedInTheFuture`, and since `validateClaims` checks `nbf` first and returns at the first failure, an issuer that sets `nbf` equal to `iat` produces a skewed token `Future` cannot help: with `nbf` and `iat` a minute ahead, `jwt.Future(2*time.Minute)` still returns `jwt: token not valid yet` while `jwt.Skew(2*time.Minute)` returns nil.

Both rescue, so both belong first, and a non-positive `d` makes `Skew` a no-op that passes the error through. `RequireExpiry` is a value rather than a function, so it takes no arguments: it returns `ErrMissingExpiry` for a token whose `Expiry` is zero, catching a `MaxAge` of zero quietly producing permanent tokens. It refuses rather than rescues, so put it after anything that rescues.

Collecting validators with Validators

```
func Validators(values ...any) []TokenValidator
```

This exists because of a Go rule that surprises people once each. `TokenValidatorFunc` implements `TokenValidator`, but a `[]TokenValidatorFunc` is not assignable to a `[]TokenValidator`: one element is a function value, the other a two-word interface value, so converting the slice would mean building a new backing array. Go declines to do that implicitly, and every call site holding one kind and needing the other grew the same copy loop:

```
jwt.Verify(alg, key, token, jwt.Validators(app.Validators, jwt.Skew(time.Minute))...)
```

It accepts a `TokenValidator`, a `TokenValidatorFunc`, a bare `func(token []byte, standardClaims Claims, err error) error`, a `[]TokenValidator` or a `[]TokenValidatorFunc`, flattens the slices and drops nils, preserving order.

The cost is in the signature: `values ...any` means the compiler checks nothing. Pass a `string`, or a typed nil matching none of the cases, and it is dropped without a word. A chain that silently lost its blocklist looks like one that never had it, so if you build validator lists dynamically, compare `len()` against what you expected.

Writing your own

A validator sees only the registered claims, so checking one of your own means decoding the payload inside the validator:

```
// ErrWrongTenant reports a token minted for a different tenant.
var ErrWrongTenant = errors.New("myapp: token is for another tenant")

// TenantValidator accepts only tokens whose "tenant" claim matches Want.
type TenantValidator struct{ Want string }

// ValidateToken implements jwt.TokenValidator.
func (v TenantValidator) ValidateToken(token []byte, standardClaims jwt.Claims, err
error) error {
	if err != nil {
		return err
	}
	decoded, err := jwt.Decode(token)
	if err != nil {
		return err
	}

	var claims struct {
		Tenant string `json:"tenant"`
	}
	if err = decoded.Claims(&claims); err != nil {
		return err
	}
	if claims.Tenant != v.Want {
```

CHAPTER 9

Revoking Tokens

A signed token is a promise you cannot take back. Once it leaves your server, every service holding your verification key will accept it until its expiry passes, and nothing in the format lets you change your mind. That is the price of a credential that verifies without a round trip. This chapter covers what to do when you need to change your mind anyway: why short lifetimes are the first answer, what a blocklist costs, and the two ways people get one wrong.

Why a token cannot be taken back

Verification is a local computation. A service holding the public key checks a signature and reads some claims, and at no point does it ask you whether the token is still wanted. That is the whole appeal: no database lookup, no network call, no shared session store.

It is also the whole problem. When a user logs out, when an account is suspended, when a laptop is stolen, the tokens already issued keep working. Deleting the token from the client is a request, not a guarantee: whoever has a copy still has a valid credential.

There is no revocation in RFC 7519, and there could not be. Anything that let an issuer withdraw a token would require the verifier to ask, and a verifier that asks is a session store with extra steps.

Short lifetimes come first

The cheapest answer is to make the window small. A token that lives fifteen minutes is a credential an attacker can use for at most fifteen minutes, and no infrastructure is required to achieve that:

```
token, err := jwt.Sign(jwt.HS256, key, claims, jwt.MaxAge(15*time.Minute))
```

This is not a complete answer, and it is worth being honest about why. Fifteen minutes is a long time if the token grants a wire transfer. It is also the window during which a suspended account still works, which is difficult to explain to whoever suspended it.

What short lifetimes do is bound the damage and reduce how often you need anything else. Reach for a blocklist for the cases the window does not cover, not for every logout.

The blocklist

A blocklist is a set of tokens the server refuses even though they still verify. This package ships an in-memory one:

```
blocklist := jwt.NewBlocklist(1 * time.Hour)
defer blocklist.Close()

// On logout.
verifiedToken, err := jwt.Verify(jwt.HS256, key, token)
if err != nil {
        return err
}

if err = blocklist.InvalidateToken(verifiedToken.Token, verifiedToken.StandardClaims);
err != nil {
        return err
}

// On every subsequent request. Blocklist implements TokenValidator.
verifiedToken, err = jwt.Verify(jwt.HS256, key, token, blocklist)
// err is now jwt.ErrBlocked
```

The argument to `NewBlocklist` is the garbage collection interval, not a lifetime. Entries are removed once the token they refer to has expired on its own, because after that point the signature check rejects it anyway and the entry is dead weight. Pass zero to disable the background sweep and call `GC` yourself.

`NewBlocklistContext` takes a context if you would rather stop the goroutine that way. `Close` stops it either way, and matters more than it looks: `NewBlocklist` passes `context.Background()`, so without `Close` the goroutine and its ticker live for as long as the process. That is fine for one blocklist created at startup and wrong for one created per test or per tenant.

The rest of the surface is small:

Method	What it does
`InvalidateToken(token []byte, c Claims) error`	Adds an entry. Returns `ErrMissing` for an empty token.
`ValidateToken(token []byte, c Claims, err error) error`	The `TokenValidator` implementation. Returns `ErrBlocked` when the entry is present.
`Has(key string) (bool, error)`	Whether a key is blocked.
`Del(key string) error`	Removes an entry.
`Count() (int64, error)`	How many entries are held.
`GC() int`	Runs one sweep and returns how many entries it removed.

Choosing the key

An entry is stored under a key derived from the token, and which key you use is the decision that matters most here.

```
func DefaultBlocklistKey(token []byte, c Claims) string {
	if c.ID != "" {
		return c.ID
	}

	return BytesToString(token)
}
```

The `jti` claim when the token has one, and the whole token otherwise. **Give every token a `jti`.** The fallback works, and it keeps a complete bearer credential, signature included, in memory for as long as the entry lives. A memory dump of a process holding a hundred blocked sessions is a hundred usable tokens.

```
token, err := jwt.Sign(jwt.HS256, key, claims,
	jwt.MaxAge(15*time.Minute),
	jwt.ID(uuid.NewString()),
)
```

Replace `Blocklist.GetKey` if you want a different scheme, and call `DefaultBlocklistKey` rather than reimplementing it. That advice is not theoretical. A storage backend written against this package returned `c.ID` unconditionally, without the fallback. Every token without a `jti` therefore mapped to the empty string, so revoking one session wrote to the key `""` and the

existence check for every other session without a `jti` read from `""`. One logout blocked every session in the system.

The in-memory `Blocklist` used to fail the same way, only more quietly. `Has` refuses an empty key with `ErrMissing`, so an entry stored under one matched nothing: `InvalidateToken` returned `nil`, the entry sat in the map, and `ValidateToken` let the token straight through. A revocation that reports success and does nothing is worse than one that fails, so `InvalidateToken` now refuses an empty key and says which two things to do about it.

`GetKey` is an exported field read outside the mutex, so set it before the blocklist is in use, not while requests are arriving. The clock is different. Garbage collection reads it on every tick from a goroutine `NewBlocklist` starts before it returns, so a field could never be assigned without racing that goroutine, and it goes in through `SetClock` instead, which is safe to call at any time. Both fall back inside the type, so a zero-value `Blocklist` works rather than panicking.

Tokens with no expiry

An entry is stored with the token's expiry, and swept once that time passes. A token with no `exp` has no such time, and the obvious implementation stores a zero.

That is what this package did, and it meant `InvalidateToken` returned `nil` and the entry was deleted on the very next sweep, because a zero expiry reads as "expired in 1970". The caller was told the session was revoked. A tick later it was live again, and nothing anywhere reported a problem.

Such entries are now stored as never expiring, and `GC` skips them. That is the only answer consistent with the token: one that never expires needs a revocation that never expires.

The consequence is a memory cost you should know about. Those entries are never collected, so the blocklist grows with the number of revoked non-expiring tokens and never shrinks. If that worries you, the fix is upstream: give tokens an expiry. `RequireExpiry` on the verifying side turns the assumption into a check.

Where it sits in the validator chain

Chapter 8 covered the rule: `Verify` runs validators in order and stops at the first one that returns an error. A blocklist is a validator, so its position decides what it sees.

Put it last, after any validator that might reject the token for a cheaper reason:

```
verifiedToken, err := jwt.Verify(jwt.HS256, key, token,
        jwt.Skew(time.Minute),
        jwt.Expected{Issuer: "auth.example.com"},
        blocklist,
)
```

There is a reason beyond tidiness. A blocklist lookup may be a network call to Redis, and an expired token does not need one. Ordering the cheap checks first means a flood of expired tokens costs you nothing downstream.

`ValidateToken` also cleans up after itself. When it is the first validator and the token has genuinely expired, it removes the entry on the way past and returns `ErrExpired` unchanged, because the claim check refuses that token anyway. Revoking a token and then verifying it after expiry takes `Count` from one back to zero, which is worth knowing before it surprises you on a graph.

A distributed blocklist

The in-memory blocklist is per process. Two instances behind a load balancer do not share one, so a logout handled by instance A does not stop instance B. For anything beyond a single process you need shared storage, and this package gives you the interface rather than the implementation:

```
type TokenBlocklist interface {
        TokenValidator

        InvalidateToken(token []byte, c Claims) error
        Del(key string) error
        Has(key string) (bool, error)
        Count() (int64, error)
}
```

Accept that rather than `*jwt.Blocklist`, and an in-memory blocklist and a Redis one become interchangeable. Here is the shape of a Redis-backed implementation:

```
type RedisBlocklist struct {
        client *redis.Client
        prefix string
        Clock  func() time.Time
        GetKey func(token []byte, c jwt.Claims) string
}
```

```
get = jwt.DefaultBlocklistKey // not a reimplementation
	}

	return b.prefix + get(token, c)
}

func (b *RedisBlocklist) InvalidateToken(token []byte, c jwt.Claims) error {
	iflen(token) == 0 {
		return jwt.ErrMissing
	}

	// Let the store expire the entry rather than sweeping it yourself. A token
with no// expiry gets no TTL at all, for the reason in the previous section.var ttl
time.Duration
	if c.Expiry > 0 {
		ttl = time.Until(time.Unix(c.Expiry, 0))
		if ttl <= 0 {
			returnnil// already expired; the claim check refuses it anyway
		}
	}

	return b.client.Set(context.Background(), b.key(token, c), 1, ttl).Err()
}

func (b *RedisBlocklist) ValidateToken(token []byte, c jwt.Claims, err error) error {
	if err != nil {
		if errors.Is(err, jwt.ErrExpired) {
			_ = b.Del(b.key(token, c))
		}

		return err
	}

	blocked, err := b.Has(b.key(token, c))
	if err != nil {
		// Decide this deliberately. See below.return err
	}
	if blocked {
		return jwt.ErrBlocked
	}

	returnnil
}
```

Three things to get right, and the third is the one people argue about.

Use `DefaultBlocklistKey`. The outage described earlier came from reimplementing it.

Let the store handle expiry. A Redis TTL removes the entry for you, and no sweep is needed. A token with no expiry gets no TTL, which is the same decision the in-memory version makes.

Decide what happens when the store is unreachable. Returning the error fails closed: every request is refused while Redis is down, including from users who were never revoked. Returning `nil` fails open: everything works, and revoked sessions come back to life for the duration of the outage. Neither is correct in general. Pick one on purpose, write down why, and make sure the choice is visible in a dashboard rather than buried in a helper.

Revoking a refresh token with its access token

`SignPair` links the two halves: the refresh token's `origin_jti` names the access token's `jti`. That link exists for this chapter.

Revoking an access token on its own accomplishes little if the refresh token that mints its replacement is still good. With the link, one lookup finds the other:

```
pair, err := jwt.SignPair(jwt.HS256, key, claims, jwt.PairOptions{
       AccessOptions:  []jwt.SignOption{jwt.MaxAge(15 * time.Minute)},
       RefreshOptions: []jwt.SignOption{jwt.MaxAge(7 * 24 * time.Hour)},
})

// pair.RefreshClaims.OriginID == pair.AccessClaims.ID
```

Store the pairing when you issue it, and on revocation block both identifiers. The alternative, blocking only what the user presented, leaves whichever half they did not present working.

Summary

A JWT verifies locally, which is why it is fast and why it cannot be withdrawn. Short lifetimes are the first answer and bound the damage without any infrastructure. A blocklist covers what the window does not, at the cost of a lookup on every request and a store to keep.

Give every token a `jti`, or the entry key is the whole token and you are keeping bearer credentials in memory. Call `DefaultBlocklistKey` rather than reimplementing it, because returning the `jti` unconditionally maps every token without one to the same empty key and one logout blocks everybody.

A revocation for a token with no expiry is kept forever, because the token is valid forever. That is correct and it is a memory cost, and the fix is to give tokens an expiry rather than to shorten the revocation.

Put the blocklist last in the validator chain, so the cheap checks run first, and decide on purpose what happens when a shared store is unreachable.

Further Reading

- RFC 7519 section 4.1.7: the `jti` claim, and the uniqueness it is expected to have.
- RFC 7009: OAuth 2.0 Token Revocation: the endpoint shape for accepting a revocation request, if you are exposing one.
- RFC 9068: a profile for JWT access tokens in OAuth 2.0, including what it says about token lifetime.
- sync.RWMutex: what the in-memory blocklist uses, and why a sweep that releases the lock between reading and deleting can lose a write.

CHAPTER 10

Tokens over HTTP

A token has to travel from the client to your handler, and there are three places it can ride: a header, a query string, or a cookie. Each leaks differently, and the twenty lines that pull the token back out are the lines every service rewrites from memory. This chapter covers the extractors this package ships, a middleware that compiles, the mapping from a verification failure to a status code, and the token pair a login endpoint hands back.

Where a token lives in a request

Location	Sent by the browser on its own	What it costs you
`Authorization: Bearer ...`	No	The client must store the token and attach it
`?access_token=...`	No	Written to every access log and to browser history
`Cookie: session=...`	Yes, on every request to the origin	CSRF protection of its own

The `Authorization` header with the `Bearer` scheme is the default answer, and RFC 6750 defines it. It is sent only when the client chooses to send it, so a form submitted from another origin cannot carry it, and proxies do not log request headers by default. The cost is that whatever can read the client's storage reads the token: in a browser, any script on the page.

A query parameter is the option to reach for last. A URL is written to the access log of every proxy and web server between the client and you, and it stays in browser history and in the address bar. There are places where nothing else works: an `EventSource` connection cannot set a header, and neither can a download link the browser follows on its own. `FromQuery` exists for those, and such a token should expire before anyone reads the log.

A cookie is sent automatically, which is the convenience and also the whole problem. A form on an unrelated site can point at your endpoint and the browser attaches the cookie. That is cross-site request forgery, and no amount of signature checking catches it, because the token really is yours and really is valid. The mitigations sit outside this package: `HttpOnly`, `Secure`, `SameSite=Lax` or `Strict`, and an anti-forgery token for what `SameSite` misses.

The extractors

Four declarations cover all three locations:

```
type TokenExtractor func(r *http.Request) []byte
const BearerPrefix = "Bearer"

func FromHeader(names ...string) TokenExtractor
func FromQuery(names ...string) TokenExtractor
func FromCookie(name string) TokenExtractor
func ExtractToken(r *http.Request, extractors ...TokenExtractor) []byte
```

Each returns `nil` when the request carries no token where it looked, and `ExtractToken` returns the first non-empty result. `FromHeader()` reads `Authorization`, `FromQuery()` reads `access_token` and then `token`, and `ExtractToken` with no extractors behaves as though you passed `FromHeader()`.

```
token := jwt.ExtractToken(r)
verifiedToken, err := jwt.Verify(jwt.HS256, key, token)
```

No nil check sits between those two lines because `Verify` returns `ErrMissing` for an empty token, so an absent credential and a forged one reach the same handler rather than two. Pass extractors when a token can arrive in more than one place, as in `jwt.ExtractToken(r, jwt.FromHeader(), jwt.FromCookie("session"))`: the first one that finds something wins, and a `nil` entry is skipped rather than dereferenced.

What FromHeader gets right

Two details in the header parser are the reason it is worth using rather than rewriting. The first is that the scheme comparison ignores case: RFC 7235 section 2.1 says the scheme is case-insensitive, so `bearer`, `Bearer` and `BEARER` are the same and clients send all three, while a parser built on `strings.HasPrefix(value, "Bearer ")` rejects two of them. The second is that the value is split on the first space only:

```
scheme, token, found := strings.Cut(value, " ")
if !found {
	return []byte(value) // A bare token with no scheme. Several gateways send
this.
}
if !strings.EqualFold(scheme, BearerPrefix) {
	continue
```

```
}
if !strings.EqualFold(scheme, BearerPrefix) {
        continue
}
if token = strings.TrimSpace(token); token != "" {
        return []byte(token)
}
```

`strings.Cut` returns everything after the first space, whatever is in it. The alternative people write, `strings.Split(value, " ")` followed by taking element one, truncates at the second space and hands verification a token missing its tail. The signature then fails, the log says the signature failed, and whoever is debugging it goes to look at keys.

Three hand-written versions of this parser across two codebases disagreed about exactly that. One truncated, one did not, and the third accepted a bare token with no scheme while the others rejected it, so the same credential was valid on one service and invalid on the next. That is a missing primitive rather than three separate bugs. Accepting a bare token is itself a leniency, since some gateways strip the scheme before forwarding; if your own edge always sends `Bearer`, write a `TokenExtractor` of your own, because the type is one function.

A middleware that compiles

Extract, verify, store the claims on the request context, and read them back through a typed accessor.

```
package auth

import (
        "context"
        "net/http"

        "github.com/kataras/jwt"
)

// claimsKey is unexported and has a type of its own, so no other package can read
this
// value out of the context or overwrite it.
type claimsKey struct{}

type UserClaims struct {
        UserID string   `json:"user_id"`
        Roles  []string `json:"roles"`
}

func Middleware(key []byte) func(http.Handler) http.Handler {
        return func(next http.Handler) http.Handler {
```

CHAPTER 11

Encrypting the Payload

Chapter 1 made the point once and this chapter is what follows from it: a JWT payload is base64url text, and anyone holding the token can read it. Signing settles who wrote the bytes and hides nothing from the person carrying them. This package can encrypt the payload with AES-GCM before signing, which changes that, and this chapter covers how, what the result is not, and why the answer is usually to store an identifier and keep the data on your own side.

What signing does not do

A signature answers one question: did a holder of this key produce these exact bytes. It says nothing about who may read them. The payload of an ordinary token is public to every party that touches it, which is the client, the browser extension the client installed, whatever proxy the request crosses, and every log line the token lands in.

Encryption changes the audience. With an encrypted payload the claims are readable only by a party holding the AES key, so the client can present the token and the token still verifies, but the client cannot see what is inside. That is the whole feature, and it is narrower than it sounds: the header stays public, the token's shape stays public, and the signing key still governs who can mint one. The case where this earns its place is a token that has to carry something the holder should not read and that you cannot look up server-side: a tenant-internal identifier that leaks your customer count, a routing hint that names an internal service, an entitlement flag whose name tells the holder which feature to go looking for. The last section of this chapter argues that most of what people reach for encryption to solve is solved better by leaving it out.

GCM and the two functions it returns

```
func GCM(key, additionalData []byte) (encrypt, decrypt InjectFunc, err error)
type InjectFunc func(plainPayload []byte) ([]byte, error)
```

`GCM` builds an AES cipher in Galois/Counter Mode and returns the two halves of it as `InjectFunc` values. `InjectFunc` is the package's general hook for transforming a payload between marshaling and encoding; encryption is one use of it, and compression and redaction are others.

The key must be 16, 24 or 32 bytes, selecting AES-128, AES-192 or AES-256. Anything else comes back from `crypto/aes` as `crypto/aes: invalid key size 20`, which you should treat

as a startup failure. There is no attack here that distinguishes AES-128 from AES-256, so use 32 bytes because it costs nothing on any processor with AES instructions, and generate it from a random source rather than a passphrase:

```
encKey := jwt.MustGenerateRandom(32) // AES-256
sigKey := jwt.MustGenerateRandom(32) // HMAC
encrypt, decrypt, err := jwt.GCM(encKey, nil)
```

Keep the two keys separate. The signing key goes wherever a token has to be verified, and the encryption key goes only where the payload has to be read; those are different sets, and if one value serves both jobs then every verifier can read every payload and the encryption bought you nothing.

Signing and verifying an encrypted token

Four functions carry the `encrypt` and `decrypt` values through:

```
func SignEncrypted(alg Alg, key PrivateKey, encrypt InjectFunc, claims any, opts
...SignOption) ([]byte, error)
func SignEncryptedWithHeader(alg Alg, key PrivateKey, encrypt InjectFunc, claims any,
customHeader any, opts ...SignOption) ([]byte, error)
func VerifyEncrypted(alg Alg, key PublicKey, decrypt InjectFunc, token []byte,
validators ...TokenValidator) (*VerifiedToken, error)
func VerifyEncryptedWithHeaderValidator(alg Alg, key PublicKey, decrypt InjectFunc,
token []byte, headerValidator HeaderValidator, validators ...TokenValidator)
(*VerifiedToken, error)
```

Apart from the extra parameter they behave exactly like `Sign`, `SignWithHeader`, `Verify` and `VerifyWithHeaderValidator`. Sign options apply the same way, validators run the same way, and `VerifiedToken.Payload` holds the decrypted JSON rather than the ciphertext.

```
claims := jwt.Map{"user_id": "12345", "role": "admin"}
token, err := jwt.SignEncrypted(jwt.HS256, sigKey, encrypt, claims,
jwt.MaxAge(15*time.Minute))
verifiedToken, err := jwt.VerifyEncrypted(jwt.HS256, sigKey, decrypt, token)
```

The result still looks like a JWT, because it is one. Its header decodes to the usual `{"alg":"HS256","typ":"JWT"}`, and its middle segment decodes to 96 bytes that are neither JSON nor text.

A token verified without its decrypt function does not fail cleanly. The signature checks out, because the signature covers the ciphertext. What fails is the claims parse, and the error is the internal “payload is not JSON” one that the `Plain` validator exists to suppress. Pass `jwt.Plain`

and verification succeeds with `Payload` holding raw ciphertext and `StandardClaims` entirely empty, expiry included. That combination is a way to accept an expired token without noticing.

The order of operations

Encrypt, then sign. Verify, then decrypt.

On the way out, `signToken` in `sign.go` marshals the claims to JSON, hands the JSON to `encrypt`, and only then calls `encodeToken`, which base64url encodes the result and signs the header and payload together:

```
	if encrypt != nil {
		payload, err = encrypt(payload)
		if err != nil {
			return nil, err
		}
	}
	return encodeToken(alg, key, payload, customHeader)
```

On the way in there are two places the decryption can happen. `decodeToken` in `token.go` is the first. Its signature check, `alg.Verify(key, headerPayload, signatureDecoded)`, sits a few lines above what follows, so the payload is only decoded and decrypted once the signature has already held:

```
	payload, err = Base64Decode(payload)
	if err != nil {
		return nil, nil, nil, err
	}
	if decrypt != nil {
		payload, err = decrypt(payload)
		if err != nil {
			return nil, nil, nil, err
		}
	}
```

That is the path the key registry takes, because `Keys.ValidateHeader` returns the matched key's `Decrypt` as its third result. `verifyToken` in `verify.go` is the second: it calls `decodeToken`, and then applies the `decrypt` you passed to `VerifyEncrypted` before parsing the standard claims.

Encrypt-then-sign is the order that makes the signature cover the ciphertext. A modified ciphertext is rejected by the cheap signature check before the AES-GCM authentication tag is ever examined, and an attacker cannot move a payload between two tokens, because the signature is over these bytes in this token.

The two decryption sites are also a hazard worth naming. Call `VerifyEncryptedWithHeaderValidator` with both an explicit `decrypt` and a header validator that returns one, and the payload is decrypted twice: the second call fails with `ErrDecrypt` on plaintext that is perfectly correct. Pick one. With the registry, pass `nil` for `decrypt` and let `ValidateHeader` supply it.

Additional authenticated data

The second argument to `GCM` is additional authenticated data, and it differs from the plaintext in one important respect: it is covered by the authentication tag, it is not encrypted, and it is not stored in the ciphertext at all. Both sides supply the same bytes independently, and if they differ by one byte then `Open` fails with `ErrDecrypt`. That makes it a binding rather than a payload: it ties a ciphertext to a context.

Suppose one AES key serves several tenants, and a payload encrypted for tenant `acme` should be undecryptable in a request routed to tenant `globex`. Build the pair per tenant with `jwt.GCM(encKey, []byte("tenant=acme"))`, and a token produced by the `acme` encrypt function fails against a decrypt function built from `tenant=globex`.

The constraint that decides whether this is usable is that the verifying side has to reconstruct the same bytes without reading the payload, since the payload is what it is trying to open. So the additional data can only come from what is visible outside the ciphertext: a header field, a hostname, a path segment, a deployment constant. If the tenant name appears only inside the encrypted claims, you cannot use it. Passing `nil`, which is what the key registry does, gives plain authenticated encryption with no context binding.

The nonce and the error you do not get

GCM needs a nonce that is never reused with the same key. The encrypt function generates 12 random bytes per call and prepends them, so the segment on the wire is nonce, then ciphertext, then a 16-byte tag:

```
ciphertext := gcm.Seal(nonce, nonce, payload, additionalData)
```

Decryption slices those 12 bytes back off. A ciphertext shorter than the nonce cannot carry one, and slicing it would panic, so the length is checked first:

```
if len(ciphertext) < gcm.NonceSize() {
        return nil, ErrDecrypt
}
```

Note what that returns: not `ErrTooShort`, not a wrapped `io.ErrUnexpectedEOF`, but the same `ErrDecrypt` a failed authentication tag returns. A distinct error there would tell an attacker the difference between a truncated payload and a forged one, which is the difference between "you cut this token short" and "your key is wrong". One distinguishable error is one bit of feedback per attempt, and that is how padding oracles are built.

In the ordinary path an attacker never reaches the decryption step, because the signature check comes first. The uniform error matters where that is not true: a service holding the signing key but not the encryption key can mint tokens whose ciphertext it chooses, and the `InjectFunc` values can be called outside this package entirely.

One limit on the random nonce is worth stating in numbers. Twelve random bytes give a 96-bit nonce, and collision probability grows with the square of the message count, so NIST SP 800-38D puts the ceiling for random nonces at 2^32 invocations under one key. Four billion tokens is not a limit you meet by accident, but it is a reason to rotate the encryption key rather than treat it as permanent.

Encryption in the key registry

`Key` carries the two functions alongside the key material:

```
type Key struct {
        ID      string        // Unique identifier for this key
        Alg     Alg           // Algorithm implementation for this key
        Public  PublicKey     // Public key for verification (can be nil for HMAC)
        Private PrivateKey    // Private key for signing (required for signing)
        MaxAge  time.Duration // Optional: automatic token expiration
        Encrypt InjectFunc    // Optional: payload encryption function
        Decrypt InjectFunc    // Optional: payload decryption function
}
```

When they are set, `Keys.SignToken` and `Keys.Verify` use them without being asked: `SignToken` calls `SignEncryptedWithHeader` with `k.Encrypt`, and `ValidateHeader` hands `k.Decrypt` back to `decodeToken`. From the caller's side the code is identical to the unencrypted case, which is the argument for configuring encryption here rather than threading `InjectFunc` values through your own call sites.

In a configuration file the field is `KeyConfiguration.EncryptionKey`, a hex string. `KeysConfiguration.Load` runs it through `hex.DecodeString`, calls `GCM` with a `nil` additional data argument, and stores both halves on the `Key`. Hex rather than base64 because a 32-byte key is 64 hex characters that survive being pasted into YAML, TOML and INI without quoting rules getting involved. The decoded length still has to be 16, 24 or 32 bytes, and a 20-byte value

fails at load time rather than at the first signature. Note the `nil`: there is no way to set additional authenticated data through configuration.

Two restrictions follow from a key having encryption attached. `Key.Configuration` refuses to export it, because the `InjectFunc` values are closures and there is nothing to serialize. And `Keys.EnrichToken` refuses to enrich a token signed with such a key, because enrichment merges claims into a plain payload and the payload it has is ciphertext.

This is not JWE

RFC 7516 defines JSON Web Encryption: a five-segment compact serialization, a content encryption key wrapped per recipient, and `alg` and `enc` header fields naming the key management and content encryption algorithms. This package implements none of it. `token_jwe.go` exists and contains no code, only a comment beginning "TODO? Add support for JWS and JWE. Currently, Out of this package's scope". What you get is a three-segment JWS whose payload segment happens to be ciphertext, and four consequences follow.

Nothing else can read it. A library expecting RFC 7516 will not decrypt it, and a library expecting a plain JWT will verify the signature and then fail to parse the payload. Both ends of every exchange have to be this package, or code written against it.

The header is not encrypted. `alg`, `typ` and any `kid` you set are visible, so tenant or environment identifiers placed in a custom header are public.

The standard claims are inside the ciphertext. `exp`, `iss` and `aud` are encrypted along with everything else, so nothing without the AES key can check them. An API gateway that today rejects expired tokens at the edge stops being able to.

There is one recipient. JWE wraps a content encryption key per recipient so several parties can decrypt with different private keys. Here there is one symmetric key, so every party that can decrypt can also encrypt, and therefore forge a payload for anyone else who shares it.

What it costs

The token gets longer. Here is the same claim set both ways, measured rather than estimated:

	Payload bytes	Token bytes
`Sign`	68	172
`SignEncrypted`	96	209

The 28-byte payload difference is fixed, 12 bytes of nonce plus a 16-byte tag, regardless of how large the claims are, and base64url turns 28 raw bytes into 37 characters of token. For a cookie with a 4 KB budget that is noise. For a header on every request of a chatty API it is a number you can multiply out.

The bigger cost is key management. You now have two key lifecycles, and rotating either one means a window where tokens signed or encrypted under the old key are still in flight, which Chapter 6 covers for signing keys and which is the same problem again here. Symmetric encryption also has no equivalent of a public key, so with one key doing both jobs you cannot give a service the ability to verify without the ability to mint.

There is also a failure mode that only appears in production. Misconfigure the encryption key on one instance out of ten and that instance verifies signatures correctly, then fails every claims parse. The error is `ErrDecrypt`, `Classify` reports `KindInvalid`, the log says the token was invalid, and nothing points at the encryption key.

When to keep the data on the server instead

The usual answer is to put an identifier in the token and store the data behind it. A `sub` claim and a lookup gets you a payload with nothing to hide and no second key lifecycle; revocation that works, since changing a row takes effect on the next request; data you can change, rather than a role the user keeps for up to fifteen minutes after you remove it; and a verifier that can check `exp` and `aud` without being trusted with the contents.

The cost is one round trip to a store you almost certainly already query for the request. Tokens carry claims to avoid that round trip, and if a claim is sensitive enough to encrypt, the round trip is usually the right trade.

Reach for encryption when the lookup is genuinely unavailable: a stateless service with no data store, a token that has to survive an offline period, or a payload that must cross a party you do not control and cannot be reduced to an opaque identifier. Those exist. They are rarer than the number of systems encrypting JWT payloads suggests.

Summary

`GCM(key, additionalData)` returns an encrypt and a decrypt `InjectFunc` built on AES-GCM, with a key of 16, 24 or 32 bytes. `SignEncrypted` and `VerifyEncrypted` carry those through the ordinary signing and verification paths, and the `WithHeader` variants do the same alongside custom headers.

The order is encrypt then sign, and verify then decrypt. The encryption happens in `signToken` before `encodeToken`; the decryption happens either in `decodeToken`, when a header validator supplies the function, or in `verifyToken`, when you pass it yourself. Supplying it both ways decrypts twice and fails.

Additional authenticated data is authenticated but not encrypted and not transmitted, so both sides must derive it identically. A 12-byte random nonce is prepended to each ciphertext, and a ciphertext too short to hold one returns `ErrDecrypt` rather than a distinct error, so nothing distinguishes a truncated payload from a forged one. The registry does all of this through `Key.Encrypt`, `Key.Decrypt` and a hex-encoded `KeyConfiguration.EncryptionKey`.

None of it is JWE. It does not interoperate, the header stays public, the token grows by 37 characters, and the AES key becomes a second key management problem beside the signing one. Before taking that on, check whether an identifier in the token and the data on your server answers the same question. It usually does.

Further Reading

- RFC 7516: JSON Web Encryption: the standard this chapter is not. Section 3 shows the five-segment serialization.
- NIST SP 800-38D: the GCM specification, the nonce rules, and the 2^32 limit on random nonces under one key.
- crypto/cipher.NewGCM: what `Seal` and `Open` do with the destination slice.
- RFC 5116: authenticated encryption with associated data as an interface, where the additional data argument comes from.

CHAPTER 12

Sharp Edges

Every library has places where the obvious reading of an API is not what it does. Most documentation leaves you to find them. This chapter is the list for this package: the defaults that are surprising, the names that read the wrong way round, the fast paths that carry a hazard, and the things that were outright broken until recently. Some of these have been fixed and are recorded here because the fix changed behaviour you may have depended on. The rest are live, and you should know about them before they find you.

Names that read the wrong way round

`Leeway` is stricter, not more lenient. The word suggests tolerance for a clock that runs a little fast. What it does is reject a token that expires within the duration you give it:

```
func Leeway(leeway time.Duration) TokenValidatorFunc {
	return func(_ []byte, standardClaims Claims, err error) error {
		if err == nil {
			if standardClaims.Expiry > 0 {
				if Clock().Add(leeway).Round(time.Second).Unix() >
standardClaims.Expiry {
					return ErrExpired
				}
			}
		}

		return err
	}
}
```

`Leeway(time.Minute)` turns away a token with fifty seconds left. It is useful when a request will take a while and you want the token to still be valid at the end of it. It is not clock-skew tolerance. `Skew` is.

`SignWithHeader` replaces the header rather than merging into it. The name suggests you are adding fields alongside the standard ones. `createCustomHeader` marshals what you hand it and uses the result as the entire header:

```
func createCustomHeader(header any) ([]byte, error) {
	b, err := Marshal(header)
	if err != nil {
		return nil, err
```

```
}

	return Base64Encode(b), nil
}
```

So `SignWithHeader(HS256, key, claims, map[string]any{"kid": "k1"})` produces a token whose header is `{"kid":"k1"}` with no `alg` at all. If you use a custom header, include `alg` and `typ` yourself, or use `HeaderWithKid`, which has both fields.

`UnverifiedToken.Enrich` verifies nothing. That is what the type name says, and it is easy to skim past when the method is called `Enrich` and the package-level `Enrich` does verify. The unverified one exists for callers who have already established authenticity some other way.

Defaults that are quieter than you want

`MaxAge` of a second or less produces a token that never expires. Read it:

```
func MaxAge(maxAge time.Duration) SignOptionFunc {
	if maxAge <= time.Second {
		return NoMaxAge
	}
	// ...
}
```

`NoMaxAge` zeroes both `exp` and `iat`. So a configuration file with `max_age: 0`, or a duration that arrived as zero because a field name was misspelled in YAML, gives you permanent tokens and no error anywhere. `RequireExpiry` on the verifying side is the defence.

A zero-value `Expected` validates nothing. Every check inside it is guarded on the field being non-empty, so `jwt.Expected{}` passed to `Verify` returns nil for any token. It looks like validation at the call site. It is not.

`LoadHMAC` treats a path that does not exist as the key itself.

```
func LoadHMAC(filenameOrRaw string) ([]byte, error) {
	if fileExists(filenameOrRaw) {
		return ReadFile(filenameOrRaw)
	}

	return []byte(filenameOrRaw), nil
}
```

That is convenient when you want to accept either form. It also means a typo in a path turns the path into your HMAC secret: `/etc/secrets/jwt.key` is twenty bytes of guessable ASCII, and

nothing errors. If the key comes from configuration, read the file yourself and fail when it is not there.

A token with no `exp` is valid forever. RFC 7519 makes the claim optional and this package follows the RFC. `RequireExpiry` is one line and worth adding.

The package-level variables

Five exported variables change behaviour for the whole process:

Variable	Default	Read on
`Clock`	`time.Now`	every claim validation
`CompareHeader`	`compareHeader`	every verification
`ReadFile`	`os.ReadFile`	every key load
`Marshal`	a function that passes `[]byte` through	every sign
`Unmarshal`	`defaultUnmarshal`	every claim decode

They are useful. `Clock` makes time-dependent tests deterministic, `ReadFile` lets keys come from an embedded filesystem, and `Unmarshal` can be swapped for a faster codec.

They are also read without any synchronisation, on paths that run concurrently. The documented rule is to set them during startup, before any goroutine verifies anything, and nothing enforces it. Assigning `jwt.Clock` from a test that runs in parallel with another test that verifies a token is a data race, and the race detector will say so.

`CompareHeader` deserves singling out. It is the algorithm check. Any package in your dependency graph can replace it for your entire process, and nothing will report that it happened.

There is a second trap that has nothing to do with races. Anything that captures one of these at construction time, rather than calling through the variable, will not see a later change:

```
// This reads the variable once, when the middleware is built.
unmarshal := jwt.Unmarshal
```

A later `jwt.Unmarshal = something` has no effect on that middleware. That is a real pattern from a real codebase, and it took a while to work out why the replacement was being ignored.

Keys is a plain map

`Keys` is `map[string]*Key`. `Register` writes to it with no lock, and `Get` and `ValidateHeader` read from it on the verification path. Building the registry at startup and reading it afterwards is fine. Registering a key while requests are in flight is a concurrent map write, which is not a race the runtime tolerates: it is `fatal error: concurrent map read and map write`, and it takes the process down with no recovery.

This is deliberate. Putting a mutex inside `Keys` would put a lock acquisition on every verification to serve the minority of callers who rotate at runtime. `KeySet` from chapter 7 is the type for that case: it swaps the whole map atomically, so readers take no lock at all.

If you need to add a key to a plain `Keys` at runtime, build a new map and swap the reference under your own synchronisation, or use `KeySet`.

Merge splices JSON and can emit a duplicate key

`Merge` is what combines your custom claims with the standard ones. It does not unmarshal and remarshal. It takes each value, checks that it is a JSON object, strips the outer braces and joins the insides with commas:

```
inner := jsonBytes[1 : len(jsonBytes)-1]
parts = append(parts, inner)
```

That is why signing is fast. It is also why merging `{"role":"admin"}` with `{"role":"user"}` produces `{"role":"admin","role":"user"}`, which is legal JSON that different parsers read differently. Go's `encoding/json` takes the last occurrence. Several other languages take the first. A token signed by this package can therefore mean one thing to your Go service and another to a Python service reading the same bytes.

The practical rule is to not restate a claim you have already set. If you need to override one, set it once in the value that wins for your reader, and be aware that a different reader may disagree.

Raw `string` and `[]byte` values are spliced in as they are, so they are checked with `json.Valid` first and rejected with `ErrNotJSONObject` otherwise. Before that check existed, a fragment like `{"broken"}` was spliced in happily and the result was a properly signed token whose payload is not JSON.

The unsafe string conversion

`BytesToString` converts a byte slice to a string without copying:

```
func BytesToString(b []byte) string {
	return unsafe.String(unsafe.SliceData(b), len(b))
}
```

The returned string shares memory with the slice. If the slice changes, the string changes with it, which is not something a Go string is ever supposed to do. Build with `-tags safe` to get the allocating version instead.

The place this matters most is the blocklist. Without a `jti`, the entry key is the whole token, produced by exactly this conversion. Storing that string as a map key would mean a caller who reuses the buffer, which is what a pooled HTTP read buffer does, silently rewrites the contents of a live key. `InvalidateToken` therefore copies the key with `strings.Clone` before storing it, and the lookup path keeps the zero-copy version, because a map lookup does not retain the string.

If you write your own code that keeps the result of `BytesToString` beyond the lifetime of the bytes, copy it.

What Enrich used to be

This one is worth knowing about even though it is fixed, because it explains the shape of the current API and because you may still have code written against the old one.

`Enrich` adds claims to a token by re-signing it. Until v0.2.0 it took the signing algorithm from the header of the token it was given, spliced that token's payload into the new one, reused the header verbatim, and verified nothing at all. The signature it produced was made with your private key.

So a service that enriched a token it had received would sign whatever header and payload the sender chose. `parseAlg` resolves the unsecured algorithm, so the sender could also ask for a token with no signature. There was no attack required beyond sending a token.

The current API takes the algorithm as its first argument, checks the signature before re-signing, and refuses the unsecured algorithm outright:

```
func Enrich(alg Alg, key PrivateKey, accessToken []byte, extraClaims any) ([]byte,
error)
```

The verification costs about eight allocations. If you have already verified the token and want them back, `Decode` followed by `UnverifiedToken.Enrich` skips the second check and leaves the safety argument with you. That method builds a fresh header from `alg` rather than copying the one it was handed, which is why a `kid` on its input is dropped.

Behaviour that changed in v0.2.0

If you are upgrading, these change the bytes on the wire or the errors you get back.

What changed	Before	Now
The unsecured algorithm's name	`NONE`, which no other implementation accepts	`none`, as RFC 7518 section 3.6 requires
`Enrich` and `UnverifiedToken.Enrich`	took no algorithm and verified nothing	take an `Alg` first and verify
`Expected.Audience`	required equal length and identical order	membership, per RFC 7519 section 4.1.3
A token with `exp: -1`	treated as having no expiry	expired
A JWK for a key type	carried empty `n`, `e`, `crv` members	omits members that do not apply
`Keys.JWKS()` for an ECDSA key	failed with `unsupported public key type`	works
`Keys.JWKS()` ordering	map order, different on every call	sorted by key id
`GenerateEdDSA`	declared `ed25519.PublicKey` and returned PEM	returns `[]byte`
`NewTokenPair(nil, nil)`	`{"access_token":"","refresh_token":""}`	`{}`
`BytesQuote`	exported	removed
An unknown algorithm name in a `KeysConfiguration`	silently became RS256	an error naming the key
A structured `sub` claim	became the string `map[a:1]`	`ErrTokenForm`

The `none` rename is the one most likely to affect you, and only if you were producing unsecured tokens with this package and reading them back with it. Tokens carrying the old `NONE` header will no longer verify.

Summary

The sharp edges here cluster into four kinds. Names that suggest the opposite of what the code does, and `Leeway` is the worst of them. Defaults that are quiet when they should complain, of which `MaxAge` collapsing to no expiry and `LoadHMAC` turning a path into a secret are the two that have cost people real time. Fast paths that carry a hazard, specifically the unsafe string conversion and the raw JSON splice in `Merge`, both of which are deliberate and both of which have a rule attached. And a set of behaviour changes in v0.2.0, most of which exist because the previous behaviour was wrong rather than merely different.

None of this is an argument against the package. It is the argument for reading this chapter before you are debugging at two in the morning.

Further Reading

- RFC 7518 section 3.6: the unsecured algorithm, and the spelling of its name.
- RFC 7519 section 4.1.3: the audience claim, and what a recipient is asked to check.
- unsafe.String: the conversion behind `BytesToString`, and the guarantees it does not make.
- Go maps are not safe for concurrent use: why a concurrent write to `Keys` is a fatal error rather than a race you can survive.
- json.Valid: the check `Merge` applies to raw fragments, and what it does not check.

CHAPTER 13

Testing

Code that signs and verifies tokens is hard to test badly and easy to test uselessly. The useless version stands up a real identity provider, waits for a real clock, and passes because the network happened to be up. This chapter is about the other kind: tests that run in milliseconds, fail for one reason, and keep failing until somebody fixes the thing they were written about. This package's own suite works that way, so most of what follows describes code you can read in the repository.

The suite that never opens a socket

This package has 112 test functions across 28 files. Not one of them opens a socket. There is no `httptest.NewServer`, no `net.Listen`, no `http.Get`. The whole suite finishes in about four seconds. That is not an accident of scope: half the package fetches key material over HTTPS, refreshes it on a timer, and refetches it when a token names a key identifier nobody has seen. All of that is tested, and none of it touches the network.

Three seams make it possible, and you get all three: `Clock` for the current time, `ReadFile` for PEM files on disk, and the `HTTPClient` interface for the JWKS fetch. Two of them are package-level variables. That is the trade-off. A seam you can reach without threading a parameter through six call sites is also a seam any other test in the same binary can reach at the same moment.

Controlling time

Every time-based check in the package reads one variable, declared as `var Clock = time.Now`. Replace it and `exp`, `nbf` and `iat` are all evaluated against whatever you return. Save the old value, restore it from `t.Cleanup` so the restore survives a failure partway through:

```
// at moves the package clock to a fixed instant and puts it back afterwards.
func at(t *testing.T, when time.Time) {
	t.Helper()

	previous := jwt.Clock
	t.Cleanup(func() { jwt.Clock = previous })
	jwt.Clock = func() time.Time { return when }
}
```

Call `at` once before signing and again with a later instant before verifying, and the expiry rule is under test rather than the scheduler. The package's own tests use this shape: `sign_test.go` copies the old value into `prevClock` and restores it from `t.Cleanup`, and `claims_test.go` does the same with a `defer`.

`Clock` is read without a lock. Every call to `Verify` reads it and your assignment writes it, which is a data race in the formal sense: the detector reports it and the compiler is entitled to do something surprising with it. So no test touching `Clock` may call `t.Parallel()`, and neither may anything running beside it. Go gives each package its own test binary, so a swap in your package cannot reach another package's tests, but within one package the whole file set shares the variable.

Where you can, skip the swap: a test that wants an expired token needs only an `exp` in the past, and that version is parallel-safe. Reach for the clock when the code under test computes a timestamp rather than reads one, which is what `MaxAge`, `SignPair` and the leeway validators do.

`Blocklist` is the counter-example: its clock is per-instance, so two blocklists in two parallel tests hold two clocks and never notice each other. `NewBlocklist(0)` starts one with no background collector, `b.SetClock` moves it an hour forward, and `b.GC()` then collects exactly the entries that expired in between. Prefer that shape in your own code.

`SetClock` is a method rather than a field for a reason worth copying. Automatic garbage collection reads the clock from a goroutine the constructor starts before it returns, so an exported field had no safe moment left for the assignment. The library shipped that field once and its own suite tripped `-race` on it. Any struct that hands a caller a knob and then reads it from a goroutine of its own has the same problem.

Keys for tests

The repository keeps a fixed set of key pairs under `_testfiles/`: RSA, RSA-PSS, ECDSA and Ed25519 in PEM, an HMAC secret, and one deliberately corrupt PEM for the parse failures. They load through helpers that panic rather than return an error, because a missing fixture is a broken checkout rather than a condition a test should branch on:

```
func MustLoadRSA(privateKeyFilename, publicKeyFilename string) (*rsa.PrivateKey,
*rsa.PublicKey)
func MustLoadECDSA(privateKeyFilename, publicKeyFilename string) (*ecdsa.PrivateKey,
*ecdsa.PublicKey)
func MustLoadEdDSA(privateKeyFilename, publicKeyFilename string) (ed25519.PrivateKey,
ed25519.PublicKey)
```

For HMAC there is nothing to load. `func MustGenerateRandom(n int) []byte` gives you a secret: 32 bytes for HS256, 48 for HS384, 64 for HS512, at 345 nanoseconds per call. For a fresh Ed25519 pair as PEM there is `func GenerateEdDSA() (publicPEM []byte, privatePEM []byte, err error)`. Note the return order. Public comes first, the opposite of the convention nearly every other key API in Go follows, and a test that binds them the wrong way round fails at the signing call rather than at the assignment. That pair took about 20 microseconds to generate, so one per test is fine.

Do not do the same with RSA. Generating a 2048-bit key took between 73 and 115 milliseconds across three runs of thirty on the machine named in Chapter 14. Thirty tests each generating one add three seconds to a suite that otherwise runs in four, and because key generation is a search for primes the cost is random as well as large. Load `_testfiles/rsa_private_key.pem` instead.

The keys in `_testfiles/` are published. They sit in a public repository and anybody can read them. They exist to make signatures reproducible, which is the opposite of what a signing key is normally for. Never let one reach a file a deployment might read.

Faking the network

The JWKS fetch takes its client as `type HTTPClient interface { Get(string) (*http.Response, error) }`, one method, and it is the method `*http.Client` already has. That is the whole reason the interface exists: `func FetchJWKS(client HTTPClient, rawURL string) (*JWKS, error)` and `func WithHTTPClient(client HTTPClient) RemoteKeySetOption` can both be handed something that is not a client at all. Here is a fake that answers from a canned string and remembers what it was asked for, modelled on `recordingClient` in `jwk_fetch_test.go`:

```
type recordingClient struct {
	status    int
	body      string
	requested []string
}

func (c *recordingClient) Get(url string) (*http.Response, error) {
	c.requested = append(c.requested, url)

	status := c.status
	if status == 0 {
		status = http.StatusOK
	}
	return &http.Response{StatusCode: status,
		Body: io.NopCloser(strings.NewReader(c.body))}, nil
}
```

The `requested` slice is what makes it more than a stub. A test can loop over `http://auth.example.com/keys`, `ftp://...` and `file:///etc/passwd`, check that every `FetchJWKS` call returns an error, and then assert `len(client.requested) == 0`. That last line is the interesting one: it proves the scheme was refused before a request went out rather than after.

The same fake covers cases a real server makes awkward. A body larger than `MaxJWKSSize`, which is 1 MB, is one `strings.Repeat` away, and a 401 whose body carries a forged log line is a string literal.

A fake that rotates

Rotation is the case a canned answer cannot test, because the point of rotation is that the answer changes. `keyset_test.go` has a client that can be reconfigured mid-test and counts its calls:

```
type rotatingClient struct {
	mu      sync.Mutex
	body    string
	fails   bool
	fetches atomic.Int64
}

func (c *rotatingClient) Get(url string) (*http.Response, error) {
	c.fetches.Add(1)

	c.mu.Lock()
	body, fails := c.body, c.fails
	c.mu.Unlock()

	if fails {
		return &http.Response{StatusCode: http.StatusInternalServerError,
			Body: io.NopCloser(strings.NewReader("upstream is unwell"))}, nil
	}
	return &http.Response{StatusCode: http.StatusOK,
		Body: io.NopCloser(strings.NewReader(body))}, nil
}

// serve replaces the answer, under the same lock Get reads it through.
func (c *rotatingClient) serve(body string, fails bool) { /* ... */ }
```

The mutex is not decoration. `KeySet` refreshes from its own goroutine, so the fake really is called concurrently, and the counter is an `atomic.Int64` for the same reason. With it, a rotation becomes deterministic:

```
client := &rotatingClient{}
client.serve(jwksFor(t, "key-one"), false)

keySet, err := jwt.NewRemoteKeySet("https://auth.example.com/keys",
       jwt.WithHTTPClient(client),
       jwt.WithRefreshInterval(0),    // no background timer; the test drives it
       jwt.WithMinRefreshInterval(0), // and no rate limit, so the refetch is
immediate
)
// verify under key-one, assert one fetch, then serve both kids and verify under key-
two
```

Setting both intervals to zero removes the timing dependency. The background refresh never fires and the rate limit on the unknown-key refetch is off, so every fetch in the count is one the code chose to make. Now the assertion can be exact: a known key adds no fetch, an unknown one adds precisely one.

The honest limit: a fake proves how your code handles a response, not that your provider sends that response. Keep one test somewhere that fetches the real JWKS, guard it with a build tag, and run it on a schedule rather than on every commit.

Golden tests

`golden_test.go` records the exact bytes this package emits and compares them on every run: the encoded header for each algorithm, whole tokens for the algorithms whose signatures are deterministic, JWK and JWKS documents, PEM round trips, `Merge` output, and the JSON shape of a `TokenPair`. ECDSA and RSA-PSS randomise the signature, so only the signed input is recorded, taken as `token[:bytes.LastIndexByte(token, '.')]`. Everything up to the last dot is deterministic and everything after it is not. Copy that split whenever you golden something with a random component. The file regenerates from a flag, `go test -run TestGolden -update`.

A golden whose expectation you edited to match new output proves nothing. It records what the code did the last time somebody ran it, and if the only reason the recording changed is that you ran it again, the test has stopped being a test. That failure mode is why people distrust golden files, and it is entirely avoidable.

Regenerating is fine. The rule is that you write down the before and the after. This repository keeps that record in `PASS-LEDGER.md`, one row per change with the old and new values side by side, so when a reviewer asks why `token/HS256` differs the answer is a line in a file rather than a memory.

Golden tests also record behaviour you know is wrong: `collectGolden` deliberately captures the error `GenerateJWK` returns for an ECDSA key, so fixing it shows up as a diff rather than as

silence. A golden file describes current behaviour, not correct behaviour.

Table-driven tests for the error paths

The error paths are where the interesting behaviour lives, and a dozen of them share one shape. A table keeps that readable:

```
for _, tt := range []struct {
	name  string
	token []byte
	want  error
}{
	{"empty", nil, jwt.ErrMissing},
	{"two segments", []byte("header.payload"), jwt.ErrTokenForm},
	{"oversized", bytes.Repeat([]byte("a"), jwt.MaxTokenSize+1), jwt.ErrTokenSize},
} {
	t.Run(tt.name, func(t *testing.T) {
		if _, err := jwt.Verify(jwt.HS256, secret, tt.token); !errors.Is(err,
tt.want) {
			t.Fatalf("expected %v but got: %v", tt.want, err)
		}
	})
}
```

Assert with `errors.Is`, never with `err.Error() == "jwt: invalid token form"`. The sentinel values are part of the package's API and will not change without a version bump. The strings around them are not, and several are wrapped with context the package is free to reword. A string comparison passes today and fails on an upgrade that changed nothing you care about. Where the error carries data, `errors.As` gets you the value: that is how the package checks its own `*HTTPError`, whose remote body is deliberately kept off the message and put on a field, because the body is chosen by the host you fetch from and ends up in whatever you log.

The limit of `errors.Is` is that it names the sentinel, not the call site. `ErrTokenAlg` comes back from four places in `token.go` alone. When the distinction matters, assert on something else as well: a status code, the fetch count your fake recorded, or the bytes that came out.

Testing that something is refused

A test that a forgery is rejected cannot build the forgery with `Sign`. `Sign` validates the algorithm name, refuses certain shapes outright, and always attaches a real signature. Going through it means testing the happy path with a different spelling.

Build the token by hand. `enrich_security_test.go` has a helper for exactly this:

```
func craftToken(header, payload, signature string) []byte {
	var b bytes.Buffer
	b.Write(jwt.Base64Encode([]byte(header)))
	b.WriteByte('.')
	b.Write(jwt.Base64Encode([]byte(payload)))
	b.WriteByte('.')
	b.WriteString(signature)
	return b.Bytes()
}
```

The header and payload are strings you write out in full, so the test says precisely what the attacker sent:

```
// The attacker presents a token claiming the unsecured algorithm and no signature.
forged := craftToken(`{"alg":"NONE","typ":"JWT"}`, `{"sub":"attacker"}`, "")

_, err := jwt.Enrich(jwt.HS256, secret, forged, jwt.Map{"role": "admin"})
if !errors.Is(err, jwt.ErrTokenAlg) {
	t.Fatalf("expected ErrTokenAlg but got: %v", err)
}
```

The same helper builds a token naming a key identifier that will never exist, which is how `keyset_test.go` checks that twenty such tokens trigger one refetch rather than twenty. Its signature segment is the literal string `sig`, because the token is refused before anything decodes it, and writing garbage there documents that fact. Three shapes belong in any suite that accepts outside tokens: an unsigned token claiming `NONE`, a token signed correctly with the wrong key, and a token whose payload was edited after signing.

Replacing Marshal and Unmarshal

Serialization goes through two more package variables. `Marshal` is a function value that passes a `[]byte` straight through and calls `encoding/json/v2`'s `Marshal` on anything else, with an option set chosen to match what `json.Marshal` produces; `Unmarshal` is `defaultUnmarshal`, an `encoding/json/v2` decode with a custom unmarshal hook that reproduces what `json.Decoder`'s `UseNumber` gave the package before Go 1.27. Replacing either follows the same save-and-restore discipline as `Clock` and carries the same parallelism warning. The usual reasons are counting how many times claims were decoded, injecting a decode failure that is otherwise hard to provoke, or checking that `jwt.UnmarshalWithRequired` really does reject a payload missing a required field.

A middleware that captured the variable at construction will not see your change. This is the trap, and it is easy to write by accident:

```
type Middleware struct{ unmarshal func([]byte, any) error }

func New() *Middleware {
        return &Middleware{unmarshal: jwt.Unmarshal} // read once, right now
}
```

`New` copies the function value out of the variable at the moment it runs. A later assignment changes the variable and leaves every already-built `Middleware` holding the old function. If your test constructs the middleware in a `TestMain` or a package-level `var`, the assignment happens after the capture and the test quietly measures nothing.

Two fixes exist and the second is better. Call `jwt.Unmarshal` at the point of use rather than storing it. Or stop reading the package variable at all: make the function an explicit field the caller sets, defaulting to `jwt.Unmarshal` when it is nil. That turns a global into a parameter, so two tests can want two different unmarshalers at once. The same reasoning applies in production: `jwt.Unmarshal = jwt.UnmarshalWithRequired` in an `init` works, and the same line in a request handler, after the middleware was built, may not.

The race detector

This package's continuous integration runs the suite twice, once plainly and once with the `safe` build tag, and both runs pass `-race`. It matters more here than in most libraries, because several types are designed to be used from many goroutines while something changes underneath them: `KeySet` replaces its key map on refresh, `Blocklist` sweeps expired entries while revocations arrive, and the package variables are read on every verification.

`TestKeySetIsSafeUnderConcurrentUse` runs eight goroutines doing two hundred verifications each while a ninth calls `Refresh` fifty times. Without `-race` that test passes whether or not `KeySet` is actually safe, because a data race usually produces correct answers on the run where you are watching.

The race detector does not run everywhere. On the machine this chapter was written on:

```
$ go test -race ./...
-race is not supported on windows/arm64
$ echo $?
2
```

The supported platform list is short and windows/arm64 is not on it. The exit code is 2, so a script that checks it does notice, which is worth confirming rather than assuming: a refusal that exited zero would let a local run report success without having checked anything. Run the race build in CI on a platform that supports it; a `linux/amd64` container substitutes fine.

Two costs. A race build runs several times slower and uses substantially more memory, which is why it belongs in CI rather than in your edit-compile loop. And the detector reports only races it observed: one in a path your test never exercised is invisible to it. It finds bugs, and it does not prove their absence.

Summary

The suite is hermetic because three seams exist: `Clock` for time, the `HTTPClient` interface for the JWKS fetch, and `ReadFile` for key files. Two are package variables read without a lock, so a test that swaps one must not run in parallel with anything else in its package, and must put the old value back through `t.Cleanup`. Load RSA keys from `_testfiles/` rather than generating them: a 2048-bit key took 73 to 115 milliseconds to generate on the machine measured, against about 20 microseconds for an Ed25519 pair. `MustGenerateRandom(32)` covers HMAC, and `GenerateEdDSA` returns the public PEM first.

Fake the network with a one-method type: a recording fake proves a URL was never requested, a rotating fake proves a key set was refetched. Golden files pin generated bytes and are worth nothing unless the regeneration is recorded where a reviewer can read it. Build forgeries by hand with `craftToken` rather than through `Sign`, assert with `errors.Is`, and run the result under `-race` on a platform that supports it.

Further Reading

- The testing package: `t.Cleanup`, `t.Helper`, `t.TempDir` and the rules that govern `t.Parallel`.
- Data Race Detector: how it works, what it costs, and the exact list of supported platforms.
- Table Driven Tests: the Go wiki page, including the argument for subtests over a bare loop.
- The errors package: `Is`, `As`, and why `%w` makes them work.

CHAPTER 14

Performance

A JWT library sits on the request path, so its cost is paid once per request forever. This package is fast for a small number of specific reasons, each of which cost something else: a precomputed header table that a custom algorithm name walks past, a zero-copy string conversion with a real aliasing hazard, and a merge that splices JSON text instead of parsing it. This chapter measures the result on one named machine, explains each decision with the price attached, and finishes with the parts that are not worth touching.

The numbers on one machine

Every figure in this chapter was measured on a Snapdragon X X1E80100, twelve cores, Windows 11, Go 1.27.0, `goos: windows` and `goarch: arm64`. Yours will differ. The ratios travel better than the absolute times.

The package's own benchmarks, from `go test -bench=. -benchmem -run=XXX` in the repository root:

```
BenchmarkSignWithMerge-12             950584     1379 ns/op    1657 B/op    22
allocs/op
BenchmarkSignWithoutMerge-12          558975     2274 ns/op    1586 B/op    30
allocs/op
BenchmarkMerge-12                    1742534      746.0 ns/op   568 B/op    10
allocs/op
BenchmarkStructToMapJSON-12           634286     1894 ns/op     762 B/op    28
allocs/op
BenchmarkStructToMapReflection-12    1000000     1285 ns/op     624 B/op    20
allocs/op
BenchmarkEnrich-12                    403982     3277 ns/op    2777 B/op    39
allocs/op
BenchmarkEncodeToken-12              1000000     1107 ns/op     888 B/op    16
allocs/op
```

The comparison suite, from the same command in `_benchmarks`:

```
BenchmarkSign_Map-12                694017     1868 ns/op    1441 B/op    27 allocs/op
BenchmarkSign_Struct-12             754218     1566 ns/op    1520 B/op    23 allocs/op
BenchmarkSign_jwt_go_Map-12         392374     2955 ns/op    2250 B/op    43 allocs/op
BenchmarkSign_jwt_go_Struct-12      614936     2512 ns/op    1986 B/op    30 allocs/op
BenchmarkSign_jwt_go_Struct2-12     667363     2185 ns/op    1826 B/op    30 allocs/op
BenchmarkSign_go_jose_Map-12        345763     5158 ns/op    4584 B/op    72 allocs/op
```

```
BenchmarkSign_go_jose_Struct-12    181594     6587 ns/op    5944 B/op    93 allocs/op
BenchmarkVerify-12                1000000     1193 ns/op     968 B/op    12 allocs/op
BenchmarkVerify_jwt_go-12          405441     3515 ns/op    2492 B/op    42 allocs/op
BenchmarkVerify_go_jose-12         163896     6320 ns/op    5896 B/op    83 allocs/op
```

Running that second command again, on the same machine and the same binary, moved `BenchmarkSign_Map` from 1883 to 2092 ns/op and `BenchmarkVerify` from 1211 to 1167. The byte and allocation columns did not move by a single unit. **Trust the allocation counts and treat the nanosecond figures as a range.** A change that removes an allocation is a real change; a change that shaves five percent off one run is probably the scheduler.

The numbers printed in `_benchmarks/README.md` are stale. They were measured years ago on an Intel i7-8750H under Windows 10, which is a different architecture at a different clock. Do not compare them with anything above.

Where the time goes in a verification

`Verify` on an HS256 token does four things: split on the dots and base64-decode the header, compare the header against a known constant, compute a MAC over the signing input and compare it against the decoded signature, then base64-decode the payload and unmarshal it.

Measuring the pieces separately, with benchmarks written for this chapter against the same token the comparison suite uses:

Piece	Time	Allocations
`Base64Decode` of one payload segment	27 ns	1
`Decode` of a whole token, all three segments	159 ns	4
HMAC-SHA256 over the signing input, fresh MAC per call	425 ns	7
`Unmarshal` of a 56-byte payload into `Claims`	427 ns	3
The whole of `Verify`	1200 ns	12

Those parts add up to more than the whole, which is the honest result and worth saying plainly: a microbenchmark of a 66-nanosecond function is measuring its own loop as much as the function, and the standalone MAC benchmark rebuilds state the real path reuses. Read the table for ordering, not for a decomposition.

The ordering is the useful part. Base64 is noise. The MAC and the JSON decode are the bill, and they are roughly the same size. The JSON decode grows with the number of claims, while the MAC grows only with the raw byte count and does so in 64-byte steps, so the one optimisation that always works is to put fewer claims in the token. Every claim you move out of the payload and into a lookup on your side comes off the decode time and the bytes on the wire together.

Precomputed headers

For every built-in algorithm the package builds three byte slices once, in `init`, and keeps them in a map:

```
type fixedHeader struct {
	raw      []byte // {"alg":"HS256","typ":"JWT"}
	encoded  []byte // the base64url form of raw
	reversed []byte // {"typ":"JWT","alg":"HS256"}
}

var fixedHeaders = make(map[string]*fixedHeader, len(allAlgs))
```

Signing looks the header up rather than building it:

```
func createHeader(alg string) []byte {
	if header := fixedHeaders[alg]; header != nil {
		return header.encoded
	}

	return Base64Encode([]byte(`{"alg":"` + alg + `","typ":"JWT"}`))
}
```

Verification never parses the header as JSON at all. `compareHeader` decodes the segment and runs `bytes.Equal` against the precomputed `raw` form, with one branch for producers that emit the fields in the other order:

```
if headerDecoded[2] == 't' {
	expectedHeader := createHeaderReversed(alg)
	// ...
}
```

That is a byte comparison against a constant instead of a JSON parse, on every request.

The cost lands in three places. A custom `Alg` whose `Name` is not in the map takes the slow path: `createHeader` concatenates and base64-encodes on every signature, and `createHeaderRaw` allocates a fresh slice on every verification. More importantly, a header this package did not produce is compared byte for byte, so a token carrying an extra header field, or

the same two fields in a third order, fails with `ErrTokenAlg` rather than being understood. There is also a length gate: `compareHeader` rejects a decoded header shorter than 25 bytes unless it is exactly 15, the length of `{"alg":"HS256"}`. Interoperating with a producer outside those shapes means supplying your own `HeaderValidator` through `VerifyWithHeaderValidator`, which parses properly and costs what parsing costs.

Bytes to string without a copy

Converting a `[]byte` to a `string` in Go allocates and copies. The package does not:

```
func BytesToString(b []byte) string {
	return unsafe.String(unsafe.SliceData(b), len(b))
}
```

The returned string points at the caller's bytes. There is a second implementation behind the `safe` build tag that is a plain `string(b)` conversion, and the continuous integration runs the full suite under both.

The hazard is aliasing, and it is not theoretical. A string in Go is immutable by contract, not by enforcement. `BytesToString` hands you one whose backing array somebody else still owns, and if they write to it your string changes underneath you. The place this bites is `Blocklist`. When a token carries no `jti`, the blocklist key is the whole token, produced by `BytesToString`, and a map key whose contents change is a map key that can no longer be found. So `InvalidateToken` copies before storing:

```
key = strings.Clone(key)
```

One allocation, on a path that runs once per logout rather than once per request. Lookups keep the zero-copy form, because a lookup does not retain the string. `TestInvalidateTokenDoesNotAliasTheCallerBuffer` hands the blocklist a buffer, overwrites every byte of it, and checks the revocation still matches.

Measure before you assume the tag is expensive. Running the comparison suite with `-tags safe` gave `BenchmarkVerify` at 1020 ns/op, 968 B/op, 12 allocs/op, which is inside the run-to-run variation of the default build for this workload. If you cannot audit every caller that hands this package a reused buffer, build with `-tags safe` and spend the difference elsewhere.

Merge splices raw JSON

`func Merge(values ...any) ([]byte, error)` combines claim sets. The obvious implementation unmarshals everything into a `map[string]any`, merges the maps, and marshals the result. This one strips the outer braces off each part and joins the insides with commas:

```
inner := jsonBytes[1 : len(jsonBytes)-1]
parts = append(parts, inner)
// ...
combined = bytes.Join([][]byte{[]byte("{"), bytes.Join(parts, []byte(",")),
[]byte("}")}, nil)
```

Never unmarshalled, never re-marshalled. The repository benchmarks the three approaches side by side, which is how the decision was made in the first place: `BenchmarkMerge` at 746 ns and 10 allocations, against 1894 ns and 28 allocations through a map, and 1285 ns and 20 allocations through reflection. End to end, `BenchmarkSignWithMerge` runs at 1379 ns and 22 allocations where `BenchmarkSignWithoutMerge` needs 2274 ns and 30.

The cost is duplicate keys. Merging `{"role":"admin"}` with `{"role":"user"}` produces `{"role":"admin","role":"user"}`. That is well-formed JSON, and RFC 8259 declines to say which value wins: Go's `encoding/json` takes the last, and other parsers take the first. The golden file records the result under `merge/duplicate-key` so nobody has to guess what the package does. The rule for callers is short. Never merge two objects that can carry the same claim, and if you cannot guarantee that, put the object whose values should win last.

One check does survive. A `string` or `[]byte` part is spliced in verbatim, so `Merge` runs `json.Valid` over exactly those two cases. Without it, `{"broken"}` passed the first-and-last byte test and produced a signed token whose payload was not JSON.

Marshal passes bytes through

The default `Marshal` starts with a type switch:

```
var Marshal = func(v any) ([]byte, error) {
	if b, ok := v.([]byte); ok {
		return b, nil
	}

	return jsonv2.Marshal(v, marshalOptions)
}
```

Since Go 1.27 the non-`[]byte` case runs on `encoding/json/v2` rather than `encoding/json`, with an option set chosen to match `json.Marshal`'s output byte for byte: sorted map keys, the same character escaping, the same leniency toward invalid UTF-8. The move to v2 buys speed without changing what gets signed.

`Merge` returns `[]byte`, `Sign` hands its claims to `Marshal`, and the payload therefore reaches the encoder without a second serialization. It also means you can cache a payload: marshal your claims once, keep the bytes, and sign them repeatedly for the price of the signature alone.

The cost is that a `[]byte` is taken at its word. `Marshal` does not check it is JSON, so `Sign(alg, key, []byte("not json"))` produces a signed token whose payload is not JSON, and the failure surfaces at whoever verifies it. `Merge` covers its own raw inputs with `json.Valid`; the direct path to `Sign` is yours to get right.

Base64Decode trims instead of padding

A JWT segment carries no base64 padding, and `base64.URLEncoding` wants input whose length divides by four. The obvious fix is to append `=` characters until it does, and this package did that for years. What replaced it goes the other way:

```
func Base64Decode(src []byte) ([]byte, error) {
	src = bytes.TrimRight(src, padStr)

	buf := make([]byte, base64.RawURLEncoding.DecodedLen(len(src)))
	n, err := base64.RawURLEncoding.Decode(buf, src)
	return buf[:n], err
}
```

Trimming rather than appending was a correctness fix first. The functions that split a token into its three parts (`bytes.Split` for a long time, `bytes.CutLast` / `bytes.Cut` since Go 1.27) do not cap the capacity of the last part they return, so the signature segment of a token still owns the token's spare capacity, and appending to it wrote past the end of the caller's buffer. With a pooled HTTP read buffer, which is how most servers hand a token to this package, that is a write into memory the next request is about to use.

It was also faster, and by more than anyone expected. `bytes.TrimRight` returns a sub-slice and allocates nothing, `RawURLEncoding` needs no padding, and `DecodedLen` on the trimmed input asks for a smaller buffer than the padded form did. The recorded before and after for `BenchmarkVerify`: 22 allocations, 1632 B and 2821 ns became 18 allocations, 1376 B and 2455 ns. Four allocations and 256 bytes off every single verification, for a change that was made to stop a memory corruption.

Measuring your own usage

The command is the one used throughout this chapter:

```
go test -bench=. -benchmem -run=XXX
```

`-benchmem` adds the two columns that matter. `-run=XXX` gives the test filter a pattern nothing matches, so the tests do not run and their time stays out of the result. Call `b.ReportAllocs()` and put `b.ResetTimer()` after any setup, or the key you generated in the benchmark body ends up in the measurement.

The counts to expect, on the small token these benchmarks use: 12 allocations to verify with HS256, 23 to sign a struct, 27 to sign a map. If yours are much higher, look at your claims type before you look at the package. A map costs more than a struct because `encoding/json` sorts the keys and boxes every value. A struct with twenty fields costs more than one with four. A `time.Time` field costs more than an `int64`.

For comparing two versions of your own code, run each with `-count=10` and put the output through `benchstat`. A single pair of runs cannot tell a five percent change from noise, and this chapter's own second run is the proof: the same binary, the same machine, 1883 ns and then 2092 ns for the same benchmark.

What not to optimise

The signature is the floor, and with anything other than HMAC it is most of the building. Verification of the same small token, by algorithm:

Algorithm	Verify	Allocations
HS256	915 ns	11
RS256	27178 ns	16
EdDSA	34546 ns	5
ES256	43500 ns	27

Signing with RS256 measured 1165527 ns, a little over 1.1 milliseconds. That is the RSA asymmetry doing its usual thing: verification uses a public exponent of 65537 and is cheap, signing uses the private exponent and is not, and the gap here is a factor of forty-three.

Read the RS256 row against the HS256 row. The package's own work is identical in both and runs on the order of a microsecond; the extra 26 microseconds is the RSA verification itself, modular arithmetic in `crypto/rsa`. Shaving another allocation off the parsing path would move an RSA verification by well under one percent, and an RSA signing by roughly nothing.

So do not tune the library for an RSA workload. Tune the number of RSA operations. Verify once per request rather than once per handler. Give access tokens a short life and a cheap algorithm and keep RSA for the refresh path. If you control both ends, EdDSA verification costs about the same as RS256 here while using five allocations instead of sixteen, and EdDSA signing measured 14989 ns against RS256's 1165527, a factor of seventy-eight.

The other thing not worth optimising is the constant-time comparison in the HMAC path. It is there so that a wrong signature takes the same time to reject as a nearly-right one, and replacing it with `bytes.Equal` would buy nanoseconds and sell an oracle.

The comparison benchmarks

`_benchmarks` compares this package against `github.com/golang-jwt/jwt` v3.2.2 and `github.com/go-jose/go-jose/v3` v3.0.4. Verification, from the run quoted above:

Package	Verify	Bytes	Allocations
`kataras/jwt`	1193 ns	968	12
`golang-jwt/jwt`	3515 ns	2492	42
`go-jose/go-jose`	6320 ns	5896	83

The second run gave 1211, 3464 and 6919 for the same three. The timing ratio is not a fixed multiple and this book will not quote one. The allocation ratio held exactly across both runs, and that is the number worth carrying: a little over a quarter of `golang-jwt`'s allocations and under a sixth of `go-jose`'s.

Two caveats belong next to those figures. `go-jose`'s `ParseSigned` does not verify anything; the benchmark has to call `Claims` afterwards to force the signature check, so the two sides are not calling equivalent APIs. And `golang-jwt/jwt` v3.2.2 is an old major version, kept because that is what the benchmark module pins. A current v5 would be a fairer comparison and has not been measured here.

Summary

Signing a struct with HS256 took about 1.6 microseconds and 23 allocations on the machine named above; verifying took about 1.2 microseconds and 12 allocations. Timings moved by roughly ten percent between two runs of the same binary while allocation counts did not move at all, so allocations are the metric to hold a change to.

Inside a verification, base64 decoding is noise and the MAC and the JSON unmarshal are roughly equal halves of the cost. Four decisions buy the rest: a precomputed header table, which a custom algorithm name and an unusual field order walk past; a zero-copy `BytesToString`, which aliases the caller's buffer and forces a `strings.Clone` in the blocklist; a `Merge` that splices raw JSON and therefore emits duplicate keys rather than resolving them; and a `Base64Decode` that trims padding, which removed four allocations from every verification and fixed a write past the end of the caller's buffer at the same time.

Above HMAC, the signature dominates everything else. RS256 verification measured 27 microseconds against HS256's 0.9, and RS256 signing 1.2 milliseconds. Optimise the number of those operations rather than the code around them.

Further Reading

- Benchmarks in the testing package: what `b.N`, `b.ResetTimer` and `b.ReportAllocs` actually do.
- benchstat: the tool for deciding whether two sets of benchmark runs differ.
- Profiling Go Programs: `-cpuprofile` and `-memprofile`, and how to read what comes out.
- unsafe.String: the exact contract, including the rule that the bytes must not be modified while the string is alive.
- RFC 8259 section 4: the paragraph that leaves duplicate object names up to the implementation.

Epilogue

The library this book describes is about four thousand lines of Go with no dependencies. It does one thing. That should make it simple, and mostly it is: `Sign` and `Verify` are two functions, and the format is three base64url segments joined by dots.

What is not simple is everything around those two functions, and that is what took most of the pages. Which algorithm, and therefore who can mint a token as well as read one. Which key, and what happens on the day it has to be replaced without a maintenance window. What a claim means when a service in another language reads the same bytes. What to do about a session that has to end before its token does.

The pattern worth taking away

Almost every defect this package has had, and there have been a number, was the same shape: something reported success while failing.

A revocation that returned `nil` and then evaporated on the next garbage collection tick. A build tag that was documented for years and had never compiled. A key set generator that returned an error for every ECDSA key, so a service published a `/.well-known/jwks.json` with a key missing from it. An expiry claim that overflowed to a negative number, failed a greater-than-zero test, and produced a token that never expired. An enrichment path that signed an attacker's header with your private key and handed back a perfectly valid token.

None of those threw. None of them showed up in a log. Each one needed somebody to sit down with the source and ask what the code does rather than what it says it does.

If you take one habit from this book, take that one. When something in an authentication path looks like it worked, check what it actually did. The failures that announce themselves are the cheap ones.

What this package is not going to do for you

It signs and verifies tokens. It does not decide who your users are, what they may do, or whether a token that verifies should be honoured. A signature proves that a holder of a key produced some bytes. Everything from there is yours: the audience check, the issuer check, the revocation list, the session model, and the decision about what a claim entitles somebody to.

It also will not stop you putting a password in a payload, granting a token a year of life, or sharing an HMAC key with a service you do not control. Chapter 12 lists the places where the library is quieter about that than it might be.

Contributing

The repository is at `github.com/kataras/jwt`. The test suite is hermetic and runs in about four seconds, so there is no excuse for a change without a test. If you find something in this book that does not match the source, the source is right and the book is wrong: open an issue and say which page.

Two things are worth knowing before you send a change. The library keeps no dependencies, and that is not negotiable; the book and the brand kit are separate modules for exactly that reason. And performance is a feature here, so a change that adds an allocation to `Sign` or `Verify` needs to say what it buys.

Thanks

To everyone who filed an issue against this package with a reproducer attached, which is rarer and more useful than it sounds. To the authors of RFC 7515, 7517, 7518 and 7519, who wrote specifications precise enough to implement from and honest enough to say where the danger is. And to the people who ran this code in production and reported what broke, which is the only way any of the defects in chapter 12 were ever found.

Go build something, and give your tokens an expiry.

Colophon

A working guide to github.com/kataras/jwt: what a JSON Web Token actually is, how this package signs and verifies one, how to choose an algorithm, rotate keys, publish and consume a JWKS, revoke a token that is still inside its lifetime, and where the sharp edges are. Every signature in this book was checked against the source.

Repository
https://github.com/kataras/jwt

Website
github.com/kataras/jwt

Licence
MIT License

Set in Inter. Rendered from Markdown to a single HTML file, then printed to PDF through a headless browser. The whole book is one file with nothing external in it: no fonts to fetch, no scripts to load, no images to go missing.